МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Практическое занятие № 1

Тема занятия: логика принятия решений в rpa-роботах
Вариант: 11

Выполнил студент 3 курса группы
ЭФБО-09-23 Калинин Н.В.

Проверил Громов Е.В.

Москва
2026 г.

Цель работы: сформировать у студентов навыки работы с данными и алгоритмами в RPA-сценариях, включая построение циклов и условий для автоматической обработки данных, формирование отчётов в различных форматах с динамическими именами файлов, корректное применение блоков Puzzle RPA для чтения, записи и обработки данных, а также понимание логики последовательности действий робота и обеспечение устойчивости сценария к разным объёмам и структурам входных данных.

В ходе выполнения практической работы студенты должны:

- изучить работу условных конструкций на простых и наглядных примерах;
- научиться реализовывать сценарии выбора в Puzzle RPA;
- получить практический опыт создания RPA-робота с использованием: условий, базовой логики обработки ошибок; понять взаимосвязь между входными данными, логикой сценария и поведением робота;
- сформировать основы алгоритмического подхода к решению задач, применимого при разработке сложных RPA-решений и автоматизации реальных бизнеспроцессов.

Результатом практической работы является разработанный RPA-сценарий, в котором робот корректно анализирует данные, принимает решения, обрабатывает нестандартные ситуации в соответствии с заданной логикой.

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

Современные RPA-решения предназначены не просто для автоматического выполнения последовательных действий, а для имитации логики работы человека в рамках бизнес-процессов. В большинстве реальных задач робот должен уметь анализировать входные данные, сравнивать значения и делать выбор между несколькими вариантами действий до достижения заданного результата.

Основой такого поведения RPA-робота является логика принятия решений, которая строится с использованием условных конструкций и базовых механизмов обработки ошибок. Именно эти элементы позволяют превратить линейный сценарий в гибкий алгоритм, способный адаптироваться к различным ситуациям и изменениям во входных данных.

Условные конструкции позволяют роботу отвечать на вопрос «что делать дальше», исходя из текущего состояния данных или результата предыдущих шагов. С их помощью реализуются сценарии выбора: выполнение одного действия при выполнении условия и другого – при его невыполнении.

В среде Puzzle RPA логика принятия решений реализуется с помощью наглядных блоков, что делает процесс разработки сценариев доступным даже для начинающих пользователей. Однако, несмотря на визуальный характер инструментов, успешная разработка RPA-роботов требует понимания алгоритмического мышления – умения разбивать задачу на шаги, определять условия переходов между ними и предугадывать возможные нестандартные ситуации.

Изучение данных тем является фундаментом для дальнейшего освоения более сложных RPA-решений, включающих многоуровневую логику, работу с большими объёмами данных и интеграцию с внешними системами.

Наша цель – научиться создавать робота, который сможет считать данные, хранить их в переменных и выполнять простую обработку, как это делает человек, но гораздо быстрее и без ошибок.

Что будем изучать:

Алгоритмы – последовательность действий для решения задачи.

Типы данных – различные виды информации, с которыми работает робот:

- Текст (*String*) – имена, слова
- Число (*Integer/Decimal*) – суммы, количество
- Дата/Время (*Date/Time*) – даты заказов
- Логический тип (*Boolean*) – *true/false*, «Да/Нет»
- Переменные – контейнеры для хранения данных и управления логикой робота: Пример: *ClientName, OrderSum, OrderDate, IsVIP*
- Логика обработки данных – как робот принимает решения на основе информации в переменных.

Сегодня вы увидите, как алгоритмы и переменные работают в реальной автоматизации, и сможете создать свой первый RPA-сценарий, который выполняет реальные действия вместо человека.

Линейный алгоритм

Для начала давайте вспомним, что такое алгоритм. Алгоритм – это чёткая последовательность действий, которая приводит к результату.

Пример из жизни: утро студента. Вы встаёте, умываетесь, завтракаете и идёте на занятия. Каждый шаг выполняется по порядку – это и есть линейный алгоритм.

В RPA робот тоже выполняет последовательность действий: открыть Excel, считать данные, обработать их и вывести результат.»

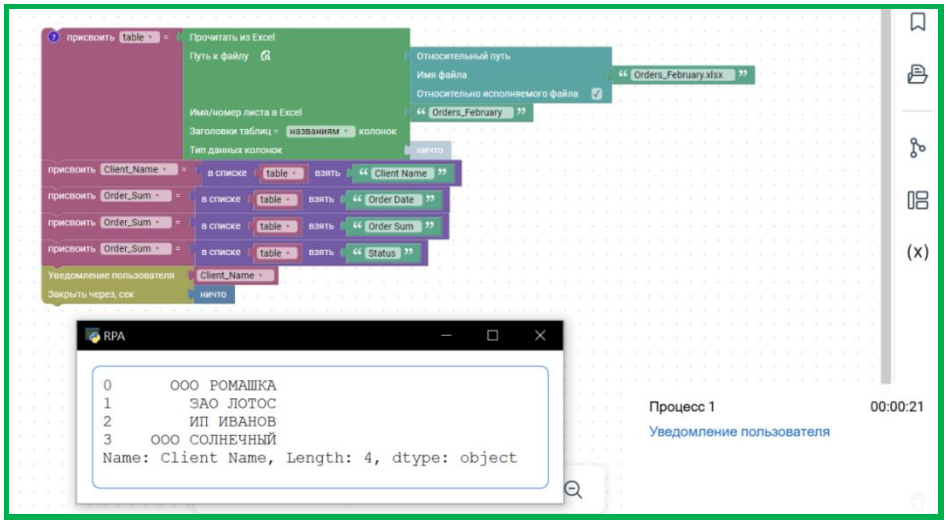
Сегодня мы начнем с линейного алгоритма. Это когда все действия идут один за другим, без условий и повторений.

Пример: робот открывает файл Excel → считывает данные → сохраняет их в переменную → выводит результат.

Важно: если один шаг пропустить, последующие действия могут не сработать. Поэтому алгоритм нужно планировать тщательно.

В Puzzle RPA это будет выглядеть как блоки, соединённые последовательно:

- Прочитать Excel-файл с заказами
- Считать данные с листа Orders_February
- Сохранить значения колонок «Клиент» ,«Сумма», «Дата», «Статус» в переменные
- Вывести уведомление со списком Клиентов.



Почему в уведомлении данные из Excel отображаются как список. Когда мы используем блок «Прочитать диапазон», Puzzle RPA: не читает одну ячейку а загружает ВЕСЬ диапазон сразу и сохраняет его в переменную типа таблица (DataTable). Это значит: Excel → превращается в таблицу, таблица состоит из строк, каждая строка состоит из ячеек. Когда ты выводишь данные в уведомлении и передаёшь туда: переменную «table», в которую попал не один элемент, а набор

данных, Puzzle RPA не знает, что именно показать, поэтому: показывает весь список значений сразу. Это нормальное поведение платформы.

Важно запомнить: Уведомление всегда показывает то, что лежит в переменной. Если в переменной таблица → показывается список. Если в переменной строка → показывается одно значение

Типы данных

Теперь давайте разберём, с чем робот работает внутри переменных. У каждой переменной есть тип данных – это определяет, что в ней хранится.

Математические операции

Инструмент работы с числовыми типами данных. Используются для расчётов и счётчиков.

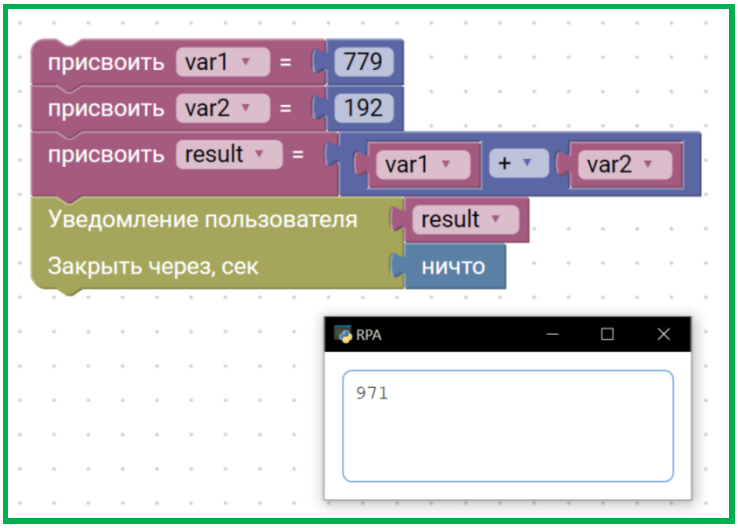
Математические операции в Puzzle RPA используются для:

- выполнения вычислений;
- работы с числовыми значениями;
- подсчёта итогов,
- количеств, сумм;
- подготовки данных перед записью в Excel;
- управления логикой алгоритма (через вычисляемые значения).

Результат любой математической операции всегда сохраняется в переменную.

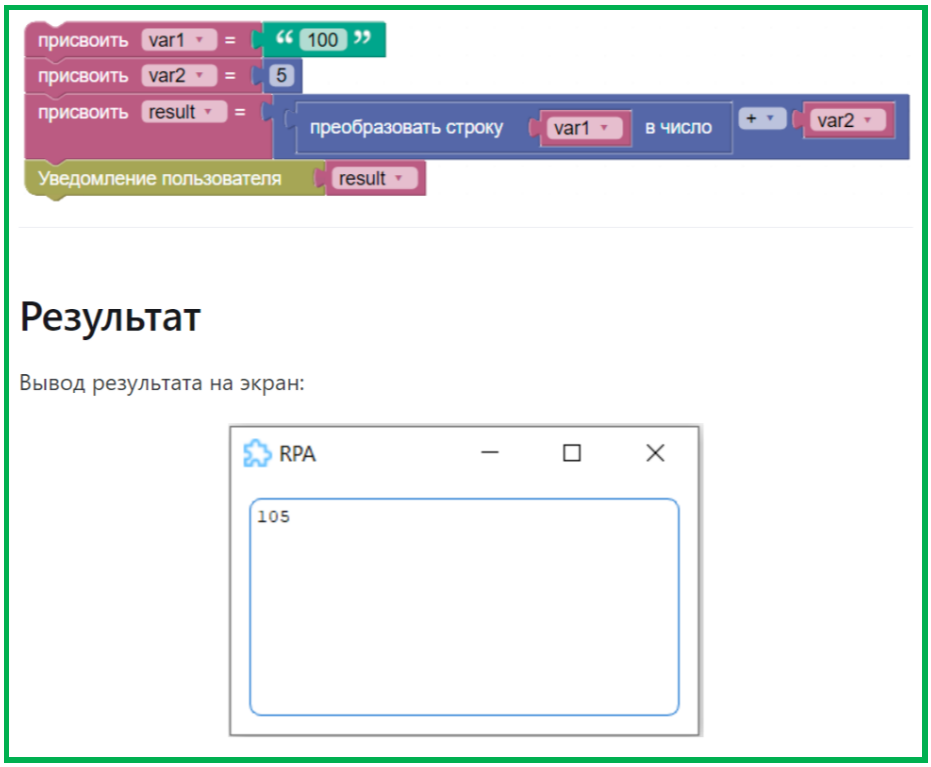
Разберём простой пример в Puzzle RPA:

- создадим переменные `var1`, `var2` и `result`,
- присвоим переменной `var1` значение 779,
- переменной `var2` значение 192,
- возьмем блок арифметических действий,
- присвоим полученный результат переменной `result`
- выведем результат через уведомление.

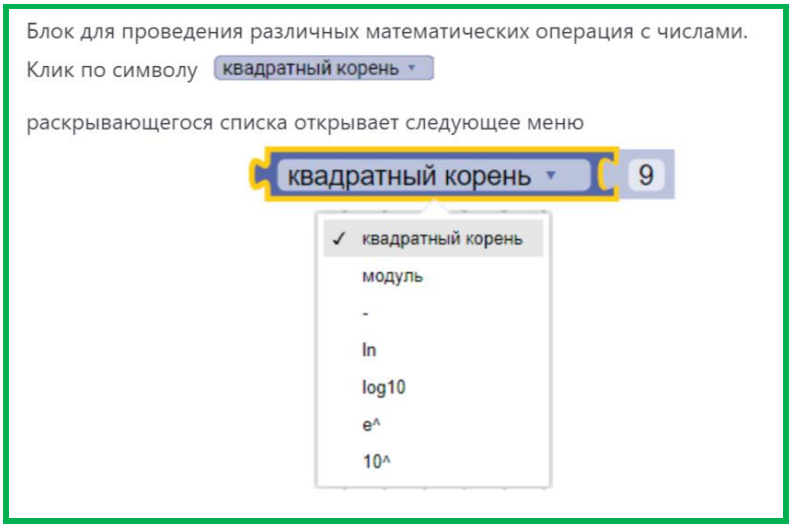


Существуют следующие типы математических операций:

- преобразовать строку в число



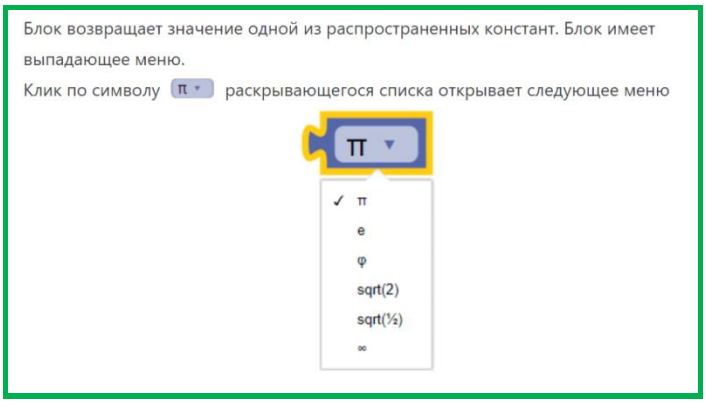
- математические операции



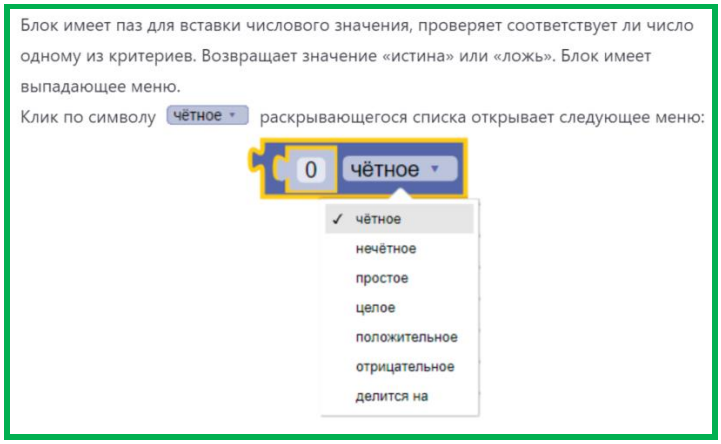
- тригонометрические операции



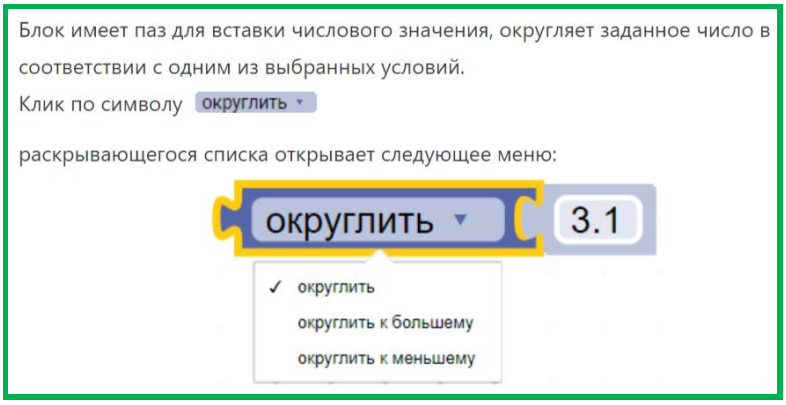
- константы



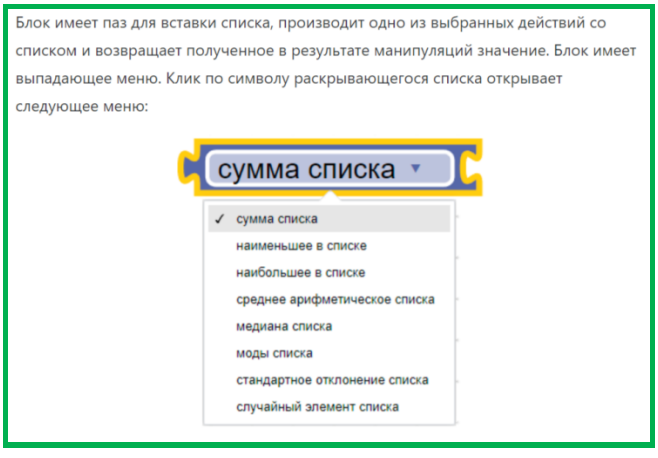
- проверка



- округлить



- математические операции со списком



Строки

Строка в Puzzle RPA – это текстовое значение, состоящее из символов (буквы, цифры, пробелы, специальные знаки). Строки используются, когда робот ра-

ботает с: именами файлов и путями; названиями листов Excel; текстовыми значениями из таблиц; сообщениями пользователю; формированием динамических данных.

Рассмотрим возможности работы со строками в Puzzle RPA:

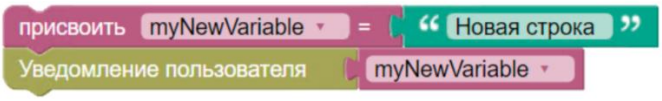
- создать строку



Блок имеет поле для ввода значений, значения внутри поля (буква, слово, строка, число, символ) будут преобразованы в строку. Если поле оставить пустым, то будет создана пустая строка.

Пример использования

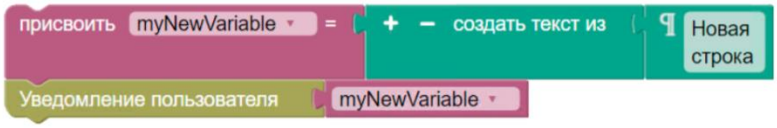
В данном примере присваивается строковое значение новой переменной **myNewVariable.**



- создать многострочную строку

Пример использования

В данном примере присваивается строковое значение новой переменной **myNewVariable.**



Блок имеет поле для ввода значений, значения внутри поля (буква, слово, строка, число, символ) будут преобразованы в строку. Если поле оставить пустым, то будет создана пустая строка. Блок позволяет добавлять в текст "\n".

"\n" - это символ перевода строки. Он обозначает начало новой строки в текстовом документе или программном коде. Когда встречается символ \n, текст, следующий за ним, будет выведен на новой строке.

- F-строка

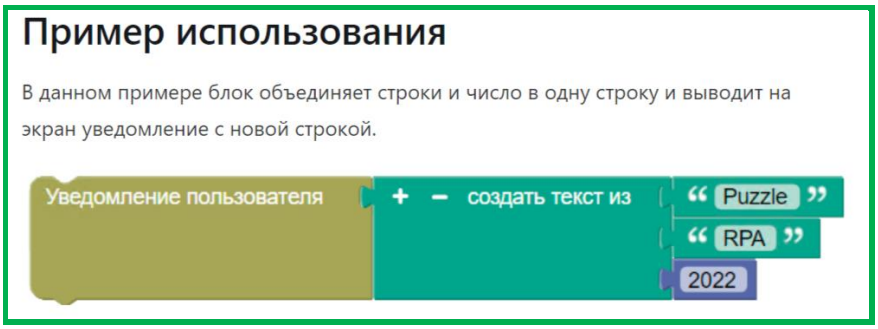


Описание

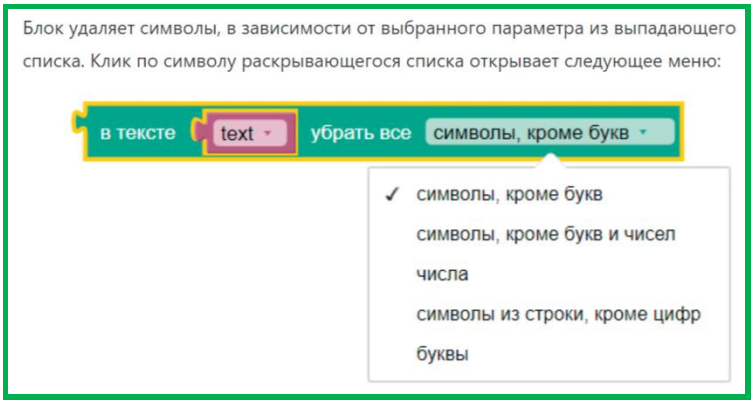
Блок имеет поле для ввода значения, которое будет преобразовано в F-строку. Если поле оставить пустым, то будет создана пустая строка.

- создать текст из

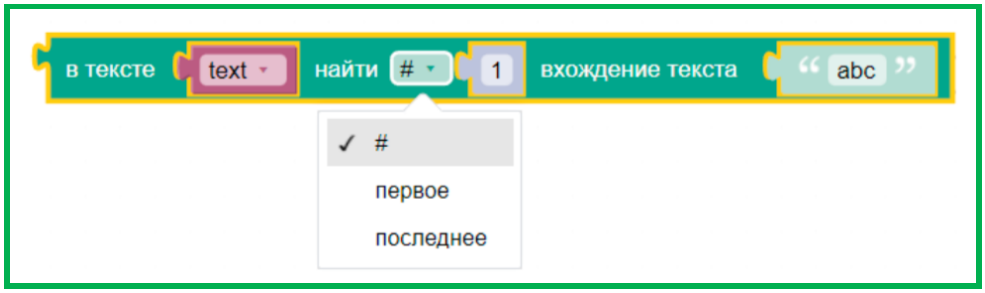
Блок объединяет присоединённые значения в одну строку. Добавляет дополнительные значения, кликнув по знаку «+».



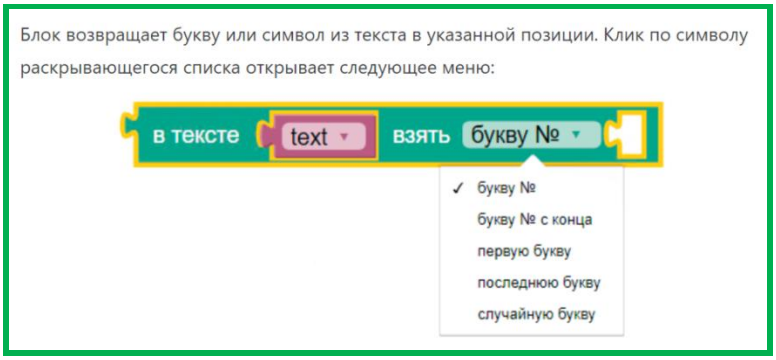
- убрать все из строки



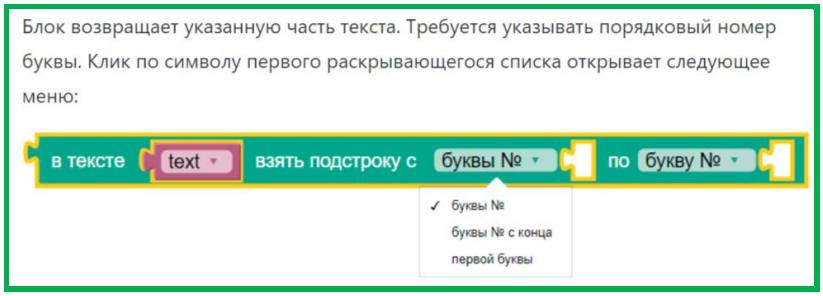
- найти вхождение текста



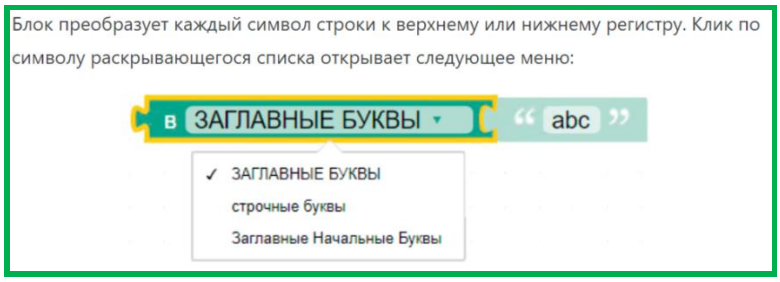
- в тексте взять



- получить подстроку



- заглавные буквы



- обрезать пробелы
- сделать текст из списка
- подсчитать количество
- изменить порядок
- напечатать

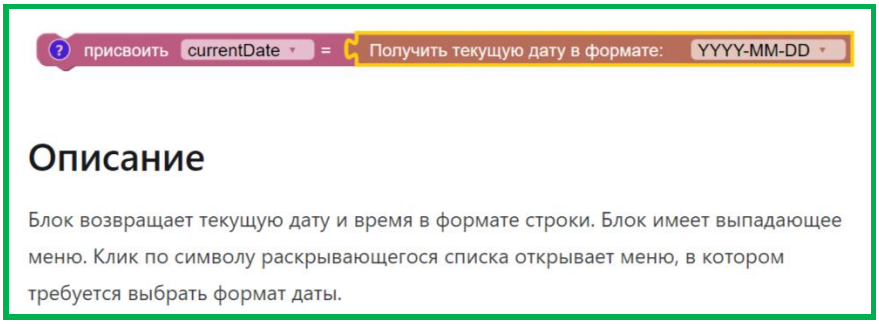
Дата

Дата – это специальный тип данных, который хранит значения времени и даты. В Puzzle RPA дата может содержать: день, месяц, год; часы, минуты, секунды (если используется формат времени); комбинированные значения дата + время. Тип данных «Дата» используется для:

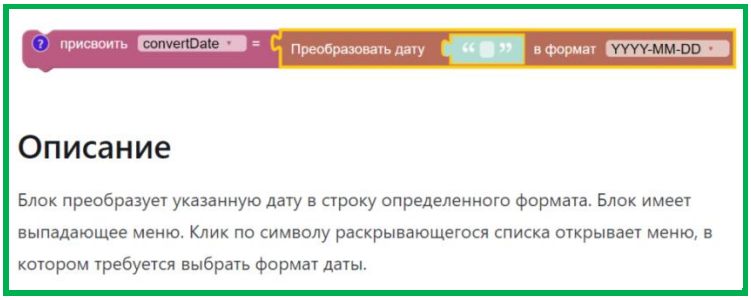
- работы с календарями и сроками;
- фильтрации и сортировки данных по дате;
- вычислений (например, разница между датами);
- автоматизации отчётов, где важны даты создания, изменения или дедлайны.

Основные блоки работы с датой

- получить текущую дату



- преобразовать дату



- вычислить дату
- установить дату
- получить текущий день недели

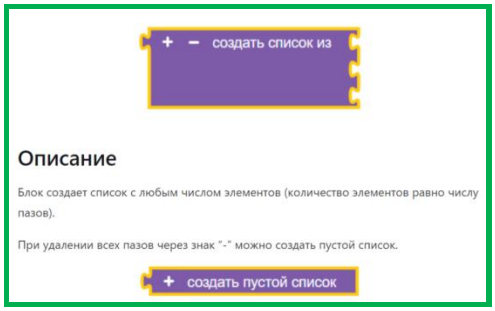
Массивы

Тип данных (List). Упорядоченный набор элементов одного типа.

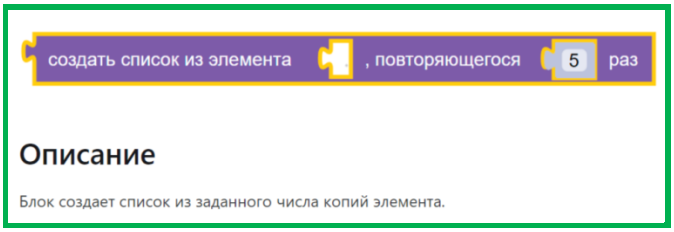
Массив в Puzzle RPA – это упорядоченный набор элементов одного типа, которые хранятся в одной переменной. Массив используется, когда: нужно хранить несколько значений, а не одно; данные приходят списком (например, строки Excel, список листов, имена файлов); требуется обрабатывать однотипные элементы.

Основные блоки работы с массивами:

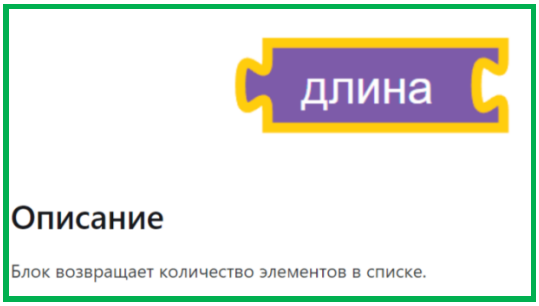
- создать список из



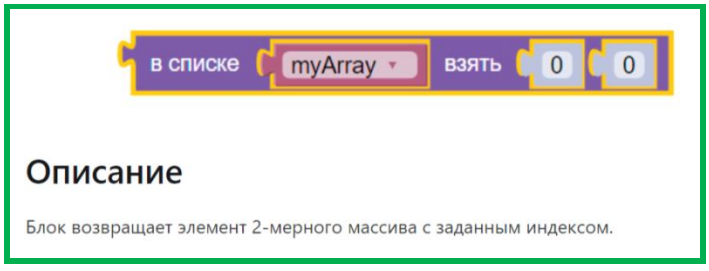
- создать список из повторяющегося элемента



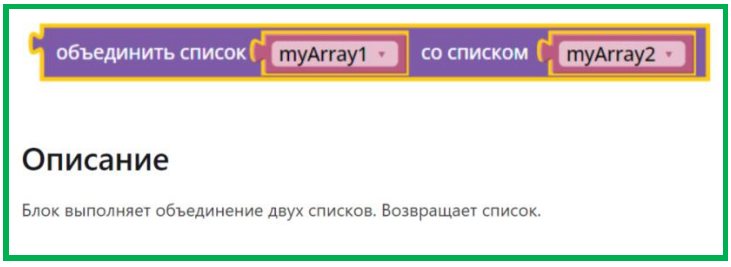
- длина списка



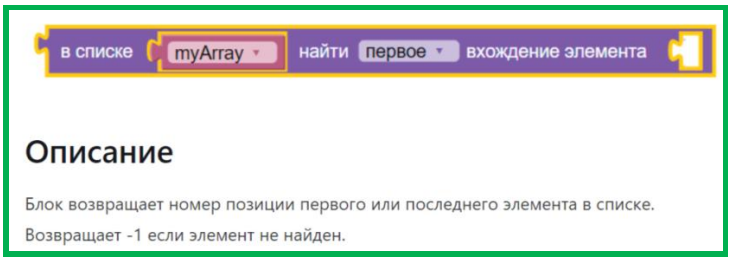
- элемент двумерного массива



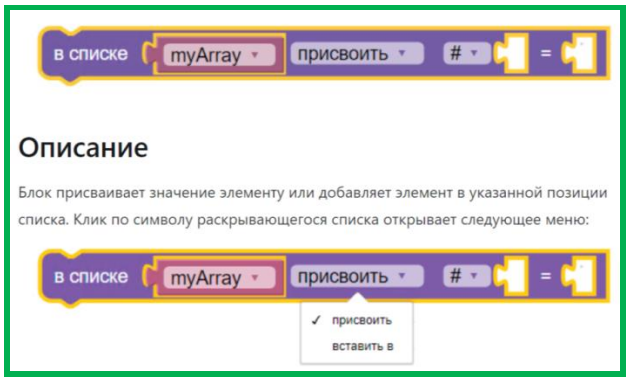
- объединить списки



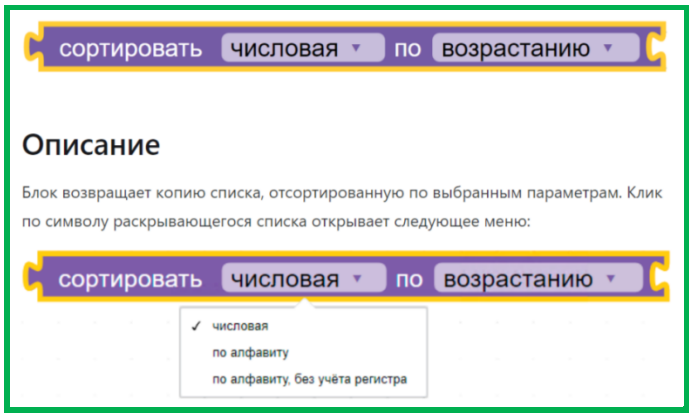
- найти номер элемента в списке



- присвоить значение элементу списка



- сортировать список



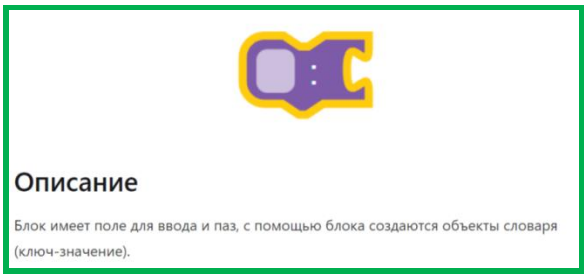
Словари

Словарь (Dictionary) – это структура данных, которая хранит пары «ключ → значение». Каждый элемент словаря имеет уникальный ключ. Ключ позволяет быстро получить соответствующее значение. Значения могут быть любого типа (строки, числа, массивы, таблицы и др.). Словари удобны, когда нужно: связывать данные между собой (например, имя сотрудника → зарплата); работать с данными, к которым нужно обращаться по ключу; структурировать информацию для дальнейшей обработки. Словари применяются:

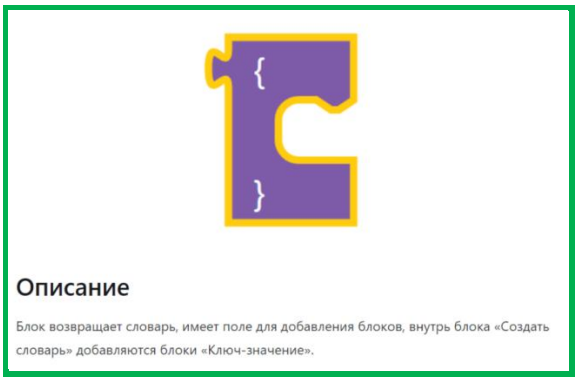
- для хранения настроек и параметров проекта;
- для сопоставления имен столбцов и значений;
- для хранения данных Excel с ключами по уникальному столбцу;
- в циклах и условиях, когда необходимо быстро искать значение по ключу.

Основные блоки работы со словарями:

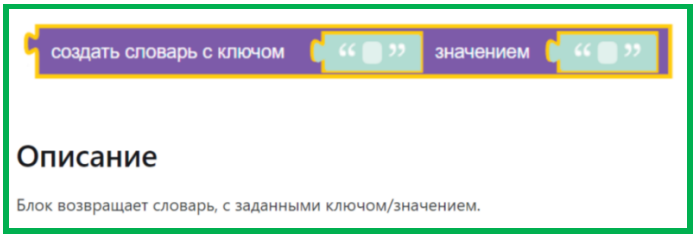
- ключ-значение



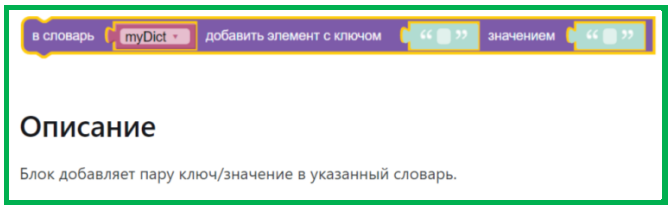
- создать словарь



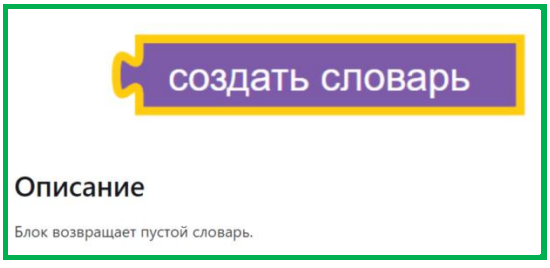
- создать словарь с ключом значением



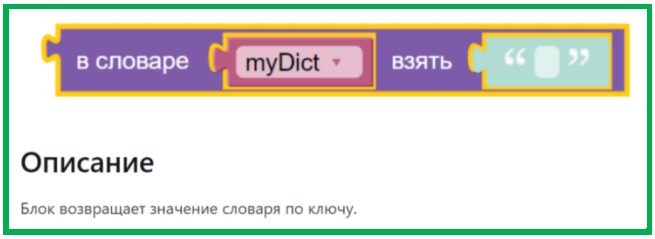
- добавить элемент в словарь



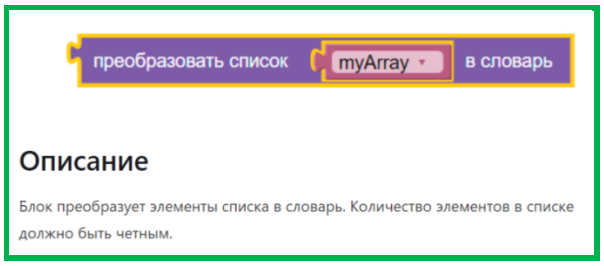
- создать пустой словарь



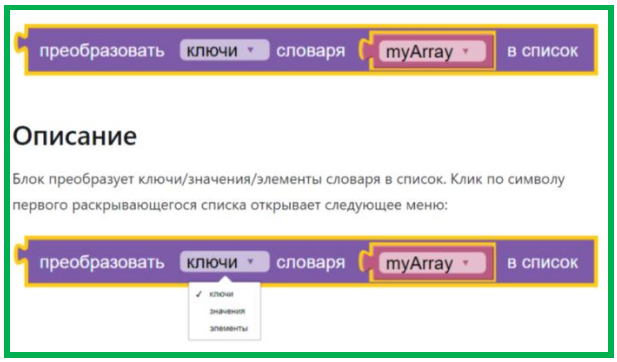
- в словаре взять



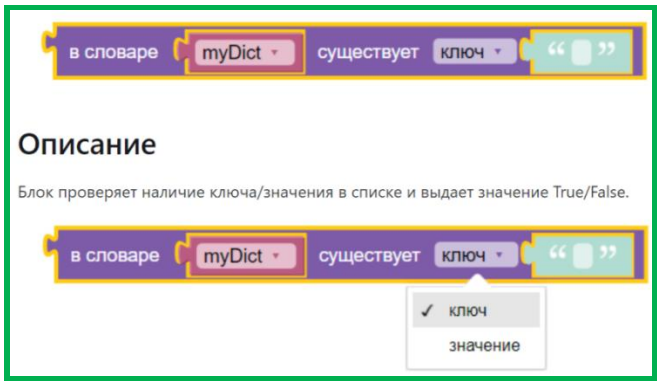
- преобразовать список в словарь



- преобразовать словарь в список



- существует ли ключ значение в словаре



Табличные данные

Табличные данные – это данные, организованные в строки и столбцы, аналогичные таблице Excel. В Puzzle RPA табличные данные хранятся в одной переменной и используются для:

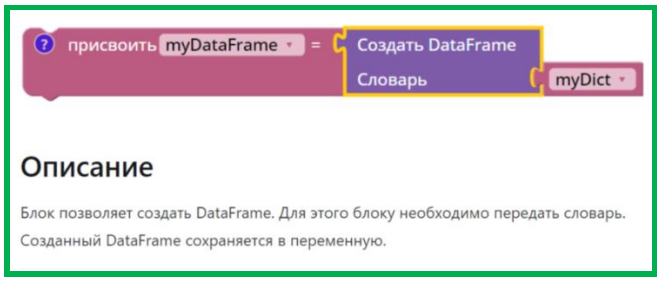
- чтения и обработки информации из Excel;
- фильтрации, сортировки и выборки строк;
- передачи данных между блоками;
- подготовки к записи в другие таблицы или системы.

Табличные данные применяются:

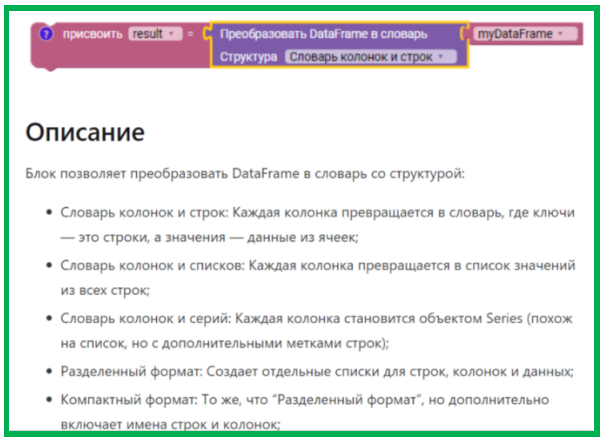
- при чтении Excel-файлов;
- при записи результатов в Excel;
- при анализе информации (например, выборка клиентов свыше 100000);
- в комбинации с массивами и словарями для обработки данных.

Основные блоки работы с табличными данными:

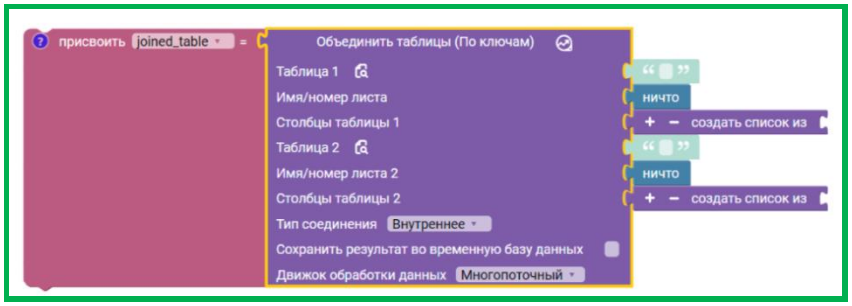
- создать DataFrame



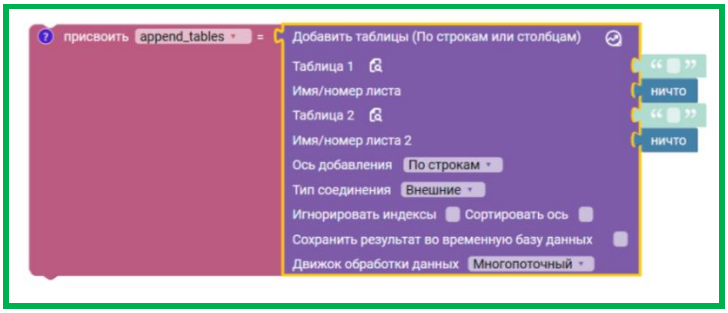
- добавить строку в DataFrame
- добавить столбец в DataFrame
- преобразовать DataFrame в словарь



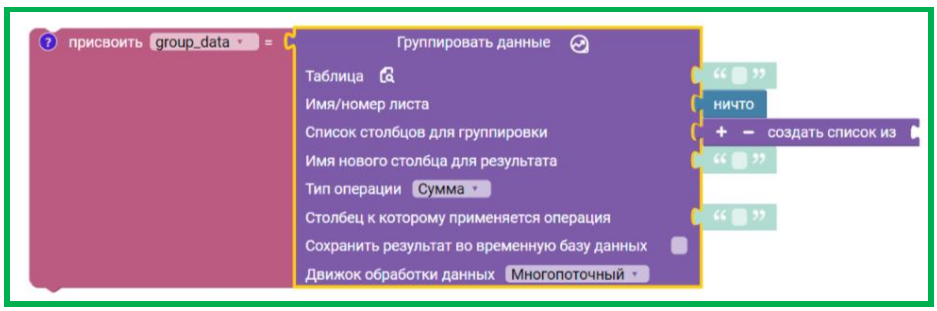
- объединить таблицы (По Ключам)



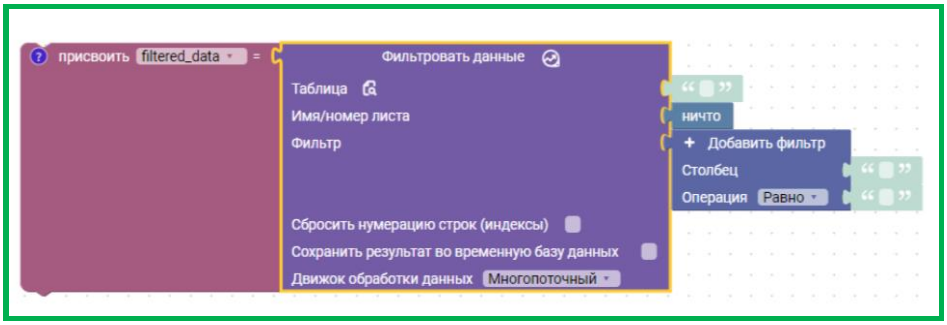
- добавить таблицы (По Строкам или Столбцам)



- группировать табличные данные



- фильтровать табличные данные



Мы подробно рассмотрели основные типы данных, которые используются в Puzzle RPA и являются фундаментом для построения любых роботов:

- Массивы – упорядоченные наборы элементов одного типа, которые позволяют хранить списки значений и обрабатывать их с помощью циклов и условий.
- Математические операции – блоки для выполнения вычислений с числами: сложение, вычитание, умножение, деление, остаток, возведение в степень, округление и др.
- Строки – текстовые данные для работы с именами файлов, листов, столбцов, сообщений и других текстовых значений.
- Словари – структуры «ключ → значение» для быстрого поиска и сопоставления данных, особенно полезны при работе с уникальными идентификаторами.
- Табличные данные – данные в виде строк и столбцов, которые хранятся в переменной и позволяют легко считывать, фильтровать, сортировать и записывать информацию в Excel.

Понимание этих типов данных позволяет: правильно строить алгоритмы роботов; хранить и обрабатывать информацию различных форматов; комбинировать данные между Excel, массивами и словарями; создавать надёжные и масштабируемые RPA-проекты.

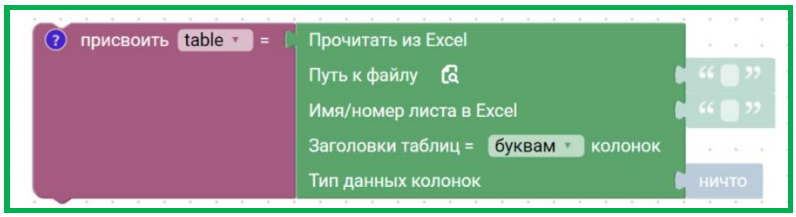
Таким образом, основы работы с типами данных – это фундамент для всех последующих практик по Puzzle RPA.

Переменные и их роль

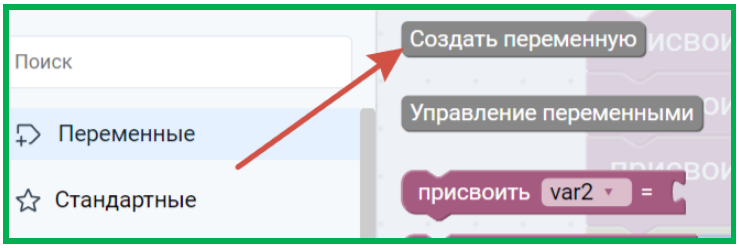
Переменная – это именованная область памяти, которая хранит значение определённого типа и используется роботом для обработки данных. Каждая переменная имеет имя, тип данных и значение. Через переменные робот передаёт данные между блоками. Переменные позволяют создавать динамические и универсальные сценарии, где данные не жестко зашиты в блоки. Основные типы переменных в Puzzle RPA мы уже рассмотрели выше.

Способы создания переменной:

- Автоматическое создание. Многие блоки Puzzle RPA создают переменные автоматически. Например, блок «Прочитать из Excel» сразу создаёт переменную типа Таблица (table).



- Ручное создание переменной. Через панель «Переменные» можно создать переменную любого типа. Присвоить начальное значение (необязательно).



Важно: имя переменной должно быть понятным и уникальным для проекта.

Переменные выполняют ключевые функции:

- Хранение данных. Робот не может «помнить» данные без переменной. Например, результат чтения Excel сохраняется в переменной `table_orders`.
- Передача данных между блоками. Значение переменной может использоваться в нескольких блоках. Пример: результат фильтрации таблицы → переменная `table_filtered` → запись в Excel.
- Динамическая обработка. Позволяет обрабатывать разные файлы, листы, строки и значения без изменения алгоритма.
- Управление логикой сценария. Переменные чисел управляют счетчиками и циклами. Переменные строк – формируют динамические пути. Переменные логики (true/false) – для условий и проверок.

Типы переменных по времени жизни:

- Локальные переменные. Создаются внутри блока или процесса. Доступны только в рамках этого блока.
- Глобальные переменные. Доступны в любом блоке процесса. Используются для передачи данных между различными этапами работы робота.

Правила именования переменных: имена должны быть понятными и отражать назначение: `table_orders` – таблица заказов; `client_name` – имя клиента; `counter` – счетчик итераций. Нельзя использовать пробелы или специальные символы (лучше нижнее подчёркивание _).

Стиль именования: `camelCase` (`clientName`) или `snake_case` (`client_name`) – единообразие важно.

Важные нюансы работы с переменными:

1. Тип переменной определяет, что с ней можно делать. Строку нельзя складывать с числом без преобразования. Таблица и массив – разные типы, нельзя напрямую присвоить одну переменную другой.

2. Переменные должны быть инициализированы. Попытка использовать переменную без значения → ошибка. Например, переменная `table_orders` должна быть заполнена блоком «Прочитать из Excel» до обработки.

3. Использование переменных в формулах и блоках Математические и логические блоки работают только с совместимыми типами. Табличные переменные передаются в блоки записи, фильтрации, выборки.

4. Совмещение переменных и массивов/словари. Массивы и словари всегда хранятся в переменных. Например: `clientsDict` – словарь всех клиентов; `ordersArray` – массив заказов. В цикле робот берёт элементы массива или словаря из переменной и обрабатывает их.

5. Переменные и относительные пути. Для работы с файлами Excel и ресурсами рекомендуется использовать переменные для путей:

```
sourceFile = "Source_Orders.xlsx"
resultFile = "Result_Orders.xlsx"
```

Так робот становится универсальным для любых проектов.

Условные конструкции в RPA-роботах

Условные конструкции используются в RPA-роботах для принятия решений на основе данных. Они позволяют роботу изменить ход выполнения сценария в зависимости от значений переменных, результатов предыдущих действий или состояния внешних систем.

Проще говоря, условие отвечает на вопрос:

«Если данные выглядят так – что должен сделать робот дальше?»

Без условных конструкций RPA-робот выполняет сценарий строго по порядку, не анализируя ситуацию. Наличие условий делает сценарий гибким и приближенным к реальным бизнес-процессам, где почти всегда существуют альтернативные варианты развития событий.

Условная конструкция строится по принципу:

ЕСЛИ – проверка условия;

ВЫПОЛНИТЬ – действия, выполняемые при выполнении условия;

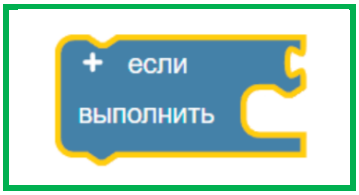
ИНАЧЕ – действия, выполняемые, если условие не выполнено.

В логическом виде это можно представить так:

Если условие истинно → выполняем один набор действий

Если условие ложно → выполняем другой набор действий

В Puzzle RPA данная конструкция реализуется с помощью блоков «Логика», которые визуально разделяют сценарий на несколько веток:



Блок проверяет условие, которое присоединено в паз «если», если условие «истинно», то выполняются блоки, вложенные в паз «выполнить». Есть возможность добавить дополнительное условие, кликнув на блоке по «+».

После клика блок изменит свой вид и появится возможность задать дополнительное условие и дополнительный набор вложенных блоков.



Блок проверяет условие, которое присоединено в паз «если», если условие «истинно», то выполняются блоки, вложенные в паз «выполнить». Если условие, которое присоединено в паз «если» - «ложь», то выполняются блоки, вложенные в паз «иначе».

Есть возможность добавить дополнительное условие, кликнув на блоке по «+». После клика блок изменит свой вид и появится возможность задать дополнительное условие и дополнительный набор вложенных блоков.

Простые условные конструкции

Простые условия включают одно логическое выражение, которое возвращает значение Истина или Ложь.

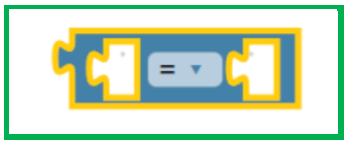
Примеры простых условий:

- сумма операции больше заданного значения;
- статус равен определённому тексту;
- значение переменной не пустое.

Пример логики:

```
если Сумма > 10000
 → обработать автоматически;
Иначе
 → отправить на проверку.
```

В Puzzle RPA простые условные конструкции реализуются также с помощью блоков «Логика»:

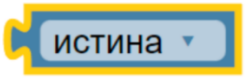


Блок сравнивает два значения, если равенство выполняется, то блок возвращает «истина». Если равенство не выполняется, то блок возвращает «ложь». Блок имеет выпадающее меню. Клик по символу раскрывающегося списка открывает следующее меню:

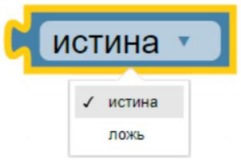


В меню представлены следующие опции:

- **=** - возвращает "истина", если значения равны;
- **≠** - возвращает "истина", если значения не равны;
- **<** - возвращает "истина", если первое значение меньше второго;
- **≤** - возвращает "истина", если первое значение меньше или равно второму;
- **>** - возвращает "истина", если первое значение больше второго;
- **≥** - возвращает "истина", если первое значение больше или равно второму.

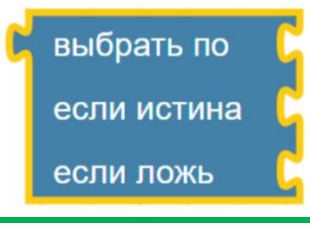


Блок присваивает переменной или выражению значение «истина». Блок имеет выпадающее меню. Клик по символу раскрывающегося списка открывает следующее меню:



В меню представлены следующие опции:

- **истина** - возвращает значение "истина";
- **ложь** - возвращает значение "ложь".



Блок проверяет условие, которое присоединено в паз «выбрать по», если условие «истинно», то выполнится блок, присоединенный в паз «если истина». Если условие, которое присоединено в паз «выбрать по» - «ложь», то выполнится блок, присоединенный в паз «если ложь».

Составные условные конструкции

Составные условия применяются в случаях, когда решение должно приниматься по нескольким параметрам одновременно. Для этого используются логические операторы:

`И` (`AND`) – все условия должны быть выполнены;

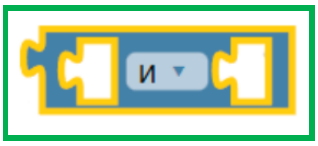
`ИЛИ` (`OR`) – достаточно выполнения одного условия;

`НЕ` (`NOT`) – инвертирует результат проверки.

Примеры:

```
сумма больше лимита
И статус = «Новый»;
тип операции = «Перевод»
ИЛИ «Оплата»;
НЕ заполнено обязательное поле.
```

Составные условия позволяют описывать более сложную бизнес-логику и часто применяются в реальных RPA-проектах. Рассмотрим примеры блоков в Puzzle RPA:



Блок проверяет два выражения, если оба «истинны», то блок возвращает «истина». Если одно или оба выражения «ложь», то блок возвращает «ложь». Блок имеет выпадающее меню. Клик по символу раскрывающегося списка открывает следующее меню:



В меню представлены следующие опции:

- **и** - возвращает "истина", если оба значения истинны;
- **или** - возвращает "истина", если хотя бы одно из значений истинно.

С помощью данного блока можно сравнивать числа, строки, переменные.



Блок возвращает значение «истина», если вставка - «ложь». Блок возвращает «ложь», если вставка - «истина».



Блок возвращает ничто (None).

Вложенные условные конструкции

Вложенные условия используются тогда, когда после выполнения одного условия требуется дополнительная проверка.

Пример логики:

```
если данные заполнены корректно:
если сумма превышает лимит
→ одна ветка;
иначе
→ другая ветка;
иначе
→ зафиксировать ошибку.
```

В Puzzle RPA вложенные условия реализуются путём размещения одного блока «Условие» внутри ветки другого условия. Такой подход позволяет пошагово уточнять логику принятия решений.

Важной частью условной логики является предварительная проверка данных. Перед выполнением основных действий робот должен убедиться, что:

- данные существуют;
- данные имеют корректный формат;
- значения подходят для дальнейшей обработки.

Примеры проверок:

- переменная не пустая;
- числовое значение больше нуля;
- строка не равна null.

Такие проверки помогают избежать ошибок выполнения сценария и являются основой устойчивых RPA-решений.

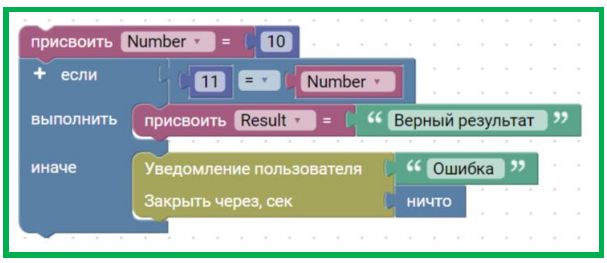
Обработка альтернативных сценариев (ELSE)

В корректно построенном сценарии всегда должна быть предусмотрена альтернативная ветка выполнения. Даже если кажется, что условие «всегда будет выполняться», практика показывает обратное.

Ветка ELSE используется для:

- фиксации ошибок;
- записи предупреждений;
- пропуска некорректных данных;
- перехода к следующему шагу сценария.

Например:



Отсутствие альтернативной ветки – одна из самых распространённых ошибок начинающих разработчиков RPA. При построении условных конструкций чаще всего возникают следующие ошибки:

- сравнение переменных разных типов данных;
- неверное логическое выражение;
- отсутствие ветки ELSE;
- проверка неинициализированных переменных;
- слишком сложная логика без пошаговой проверки.

В рамках практической работы студенты учатся анализировать и исправлять такие ошибки, используя пошаговое выполнение сценария и логирование.

Работа с условиями формирует у студентов понимание того, что:

- любой сценарий можно разложить на набор проверок;
- каждое решение должно иметь чёткое основание;
- алгоритм должен учитывать как основной, так и альтернативный путь выполнения.

Это является ключевым навыком для дальнейшей разработки сложных RPA-решений, включающих циклы, обработку исключений и интеграцию с внешними системами.

Циклы в RPA: архитектура повторяющихся процессов

В профессиональной RPA-разработке цикл – это не просто «повторение действий», а механизм:

- масштабирования алгоритма,
- обработки коллекций данных,
- построения устойчивых процессов,
- управления состоянием выполнения.

Если условия отвечают за принятие решений, то циклы отвечают за обработку объёма. Практически любой *production*-процесс в RPA содержит хотя бы один цикл:

- обработка строк DataTable,
- перебор элементов DOM,
- итеративные попытки (retry logic),
- контроль состояния до наступления события.

В Puzzle RPA применяются архитектурно разные типы циклов.

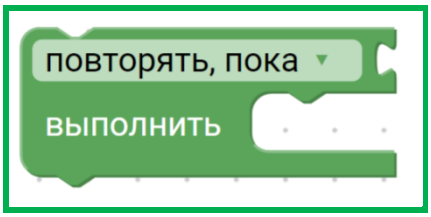
Цикл с фиксированным числом повторений (For)



Используется для: retry-механизмов, регламентированных повторов, циклических проверок.

Пример: повторить 3 раза: попытаться авторизоваться. В архитектуре RPA это часть стратегии устойчивости (resilience design).

Цикл с условием (While)



Назначение: управление состоянием процесса. Используется, когда:

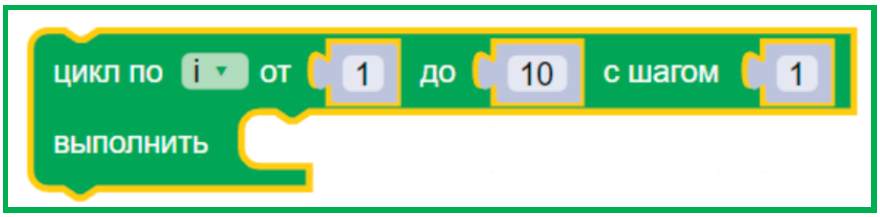
- неизвестно количество повторений,
- нужно дождаться события,
- процесс зависит от динамического условия.

Логика: пока условие истинно: выполнять действия

Ключевая профессиональная проблема – контроль выхода. Любой цикл While должен иметь:

- Условие завершения.
- Изменяемый параметр внутри цикла.
- Защитный механизм (timeout или счетчик итераций).
- Иначе возникает риск бесконечного цикла.

Цикл с диапазоном: от–до–шаг



Это разновидность цикла For, в котором:

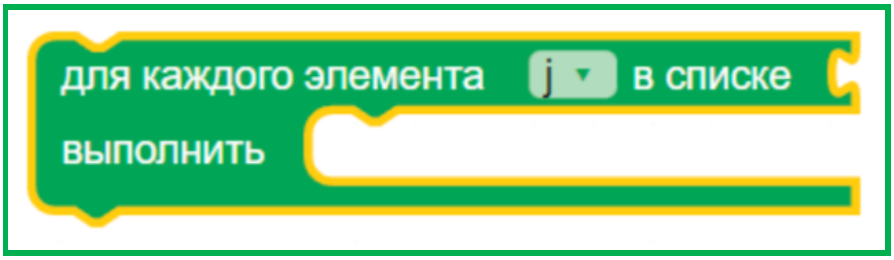
- задаётся начальное значение,
- конечное значение,
- шаг изменения,
- управляющая переменная (счётчик).

Такой цикл применяется, когда количество итераций известно заранее и требуется точный контроль индекса.

Цикл «от–до–шаг» – это управляемая числовая итерация. В отличие от цикла по коллекции:

- источник итерации – не таблица, а диапазон чисел;
- контроль осуществляется через счётчик;
- логика не зависит от структуры данных.

Цикл «Для каждого элемента» (For Each)



Цикл For Each – это механизм итерации по коллекции без явного управления индексом. Ключевое отличие от цикла «от–до–шаг»:

- нет счётчика,
- нет ручного управления границами,
- источник итерации – структура данных.

Это декларативная модель обхода, а не числовая.

Даже без счётчика, цикл For Each управляет состоянием:

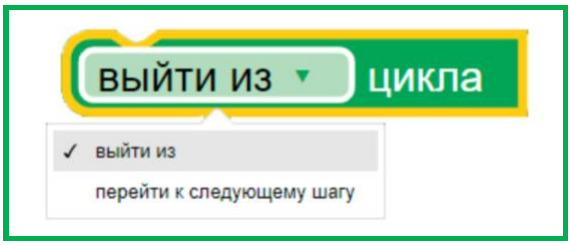
- изменяются переменные,
- накапливаются суммы,
- формируются отчёты,
- записываются результаты.

Нельзя эффективно использовать For Each, если:

- нужно пропускать элементы по индексу;
- требуется шаг > 1;
- необходимо идти в обратном порядке;
- нужно удалять элементы во время обхода.

В этих случаях предпочтительнее цикл «от–до–шаг».

Выход из цикла (Break) и переход к следующей итерации (Continue)



Внутри любого цикла существуют два механизма контроля:

- Принудительный выход из цикла
- Пропуск текущей итерации и переход к следующей

Это разные по смыслу операции.

Выход из цикла (Break). Немедленно завершает выполнение цикла независимо от условия. После выполнения команды: дальнейшие итерации не выполняются, управление передаётся следующему блоку после цикла.

Когда используется:

1. Найдено искомое значение. Нет смысла продолжать обработку.

2. Критическая ошибка. Продолжение обработки может привести к некорректным данным.

3. Достигнут лимит. Например, найдено 10 совпадений – дальше искать не нужно.

Переход к следующей итерации (Continue). Прерывает текущую итерацию и переходит к следующей. Цикл продолжается.

Когда используется

1. Фильтрация данных. Пропустить строки, которые не соответствуют критериям.

2. Неполные данные. Если в строке отсутствует обязательное поле.

3. Предварительная проверка. Отсеять ненужные элементы в начале.

Вложенные циклы

Теперь перейдем к вложенным циклам. Вложенные циклы увеличивают сложность процесса. Если: 50 проектов, в каждом 20 задач. Это уже 1000 итераций.

При взаимодействии с UI это может привести к:

- увеличению времени выполнения,
- риску таймаутов,
- перегрузке системы.

Поэтому важно:

- минимизировать вложенность,
- предварительно фильтровать данные,
- переносить часть логики на этап подготовки данных.

Частые ошибки:

- Использование Break вместо Continue.
- Отсутствие логирования при досрочном выходе.
- Break внутри вложенного цикла без понимания уровня выхода.
- Пропуск очистки переменных перед выходом.

Циклы в Puzzle RPA – это не просто механизм повторения действий. Это инструмент управления:

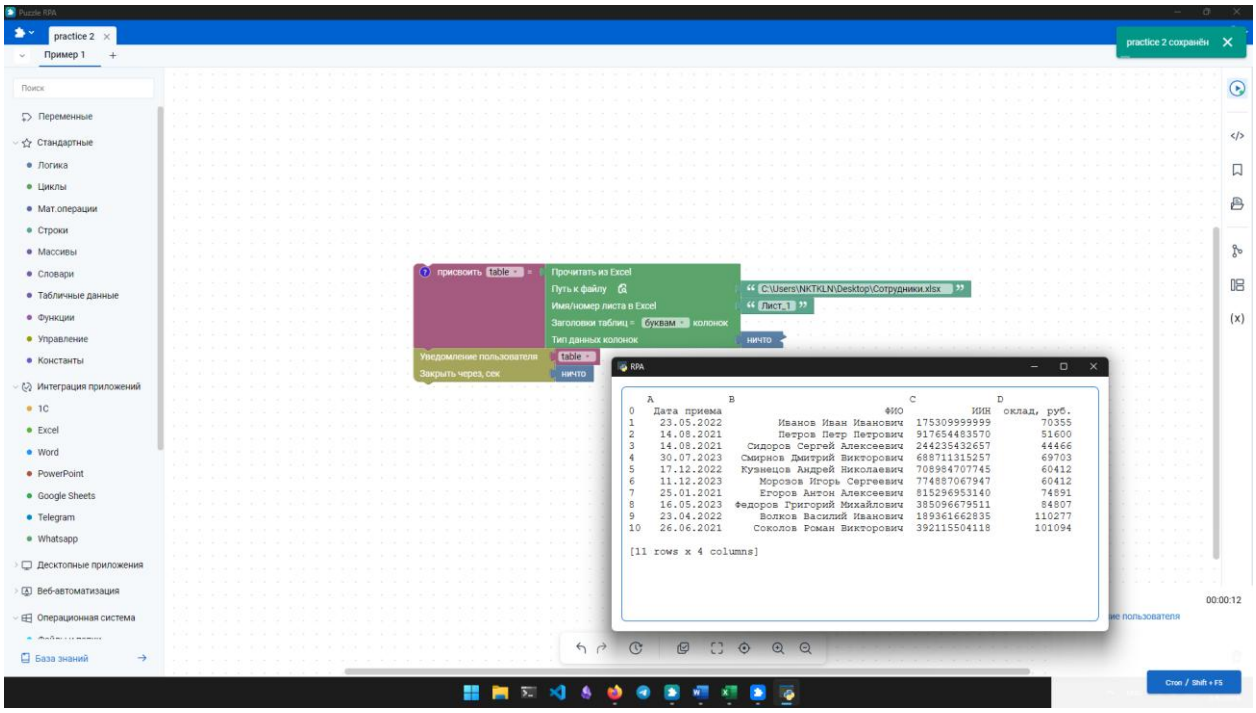
- объёмом данных,
- состоянием системы,
- сложностью алгоритма,
- производительностью процесса.

Если условия определяют логику решений, то циклы определяют масштаб применения этих решений.

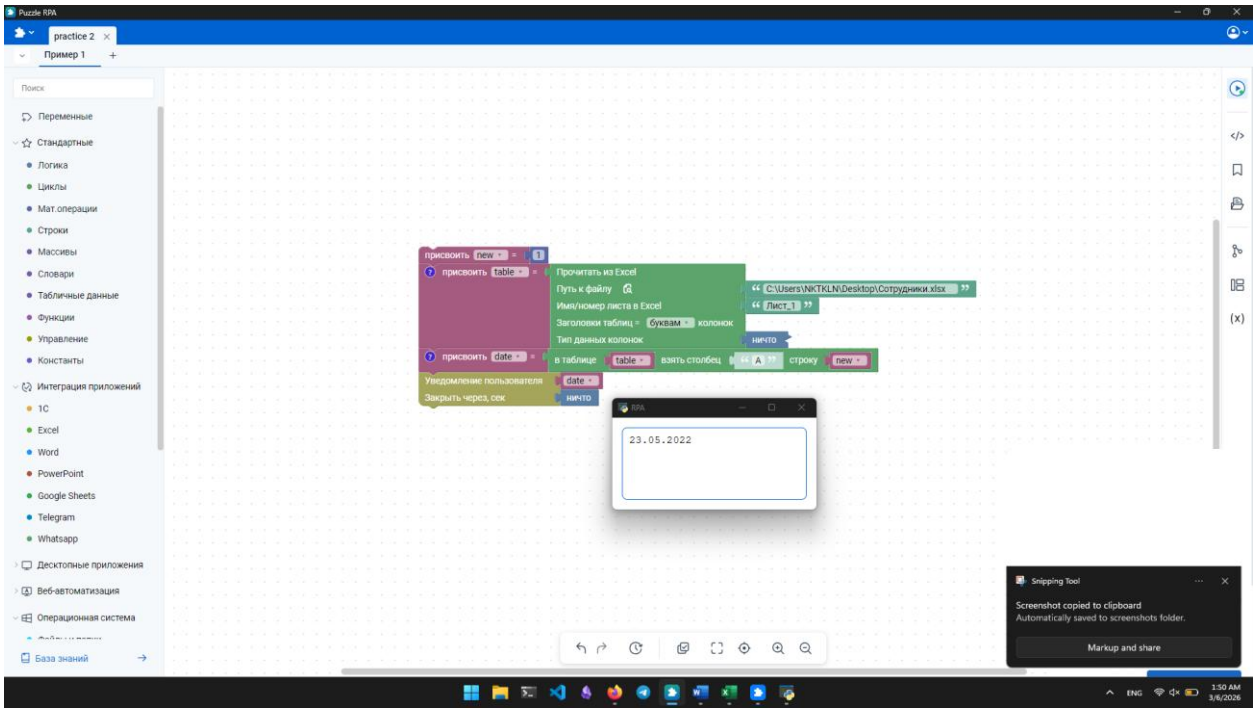
ПРИМЕРЫ ЗАДАЧ

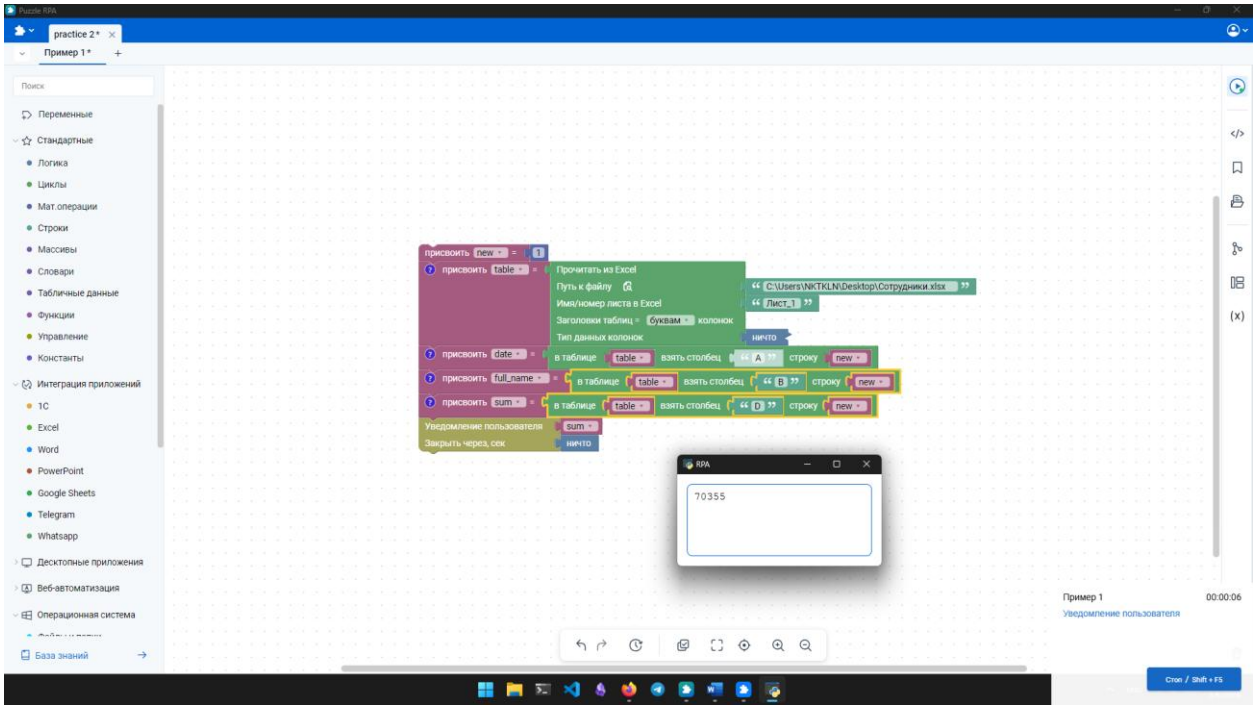
Пример 1. Прочитать Excel и сохранить данные первой строки

Чтение таблицы

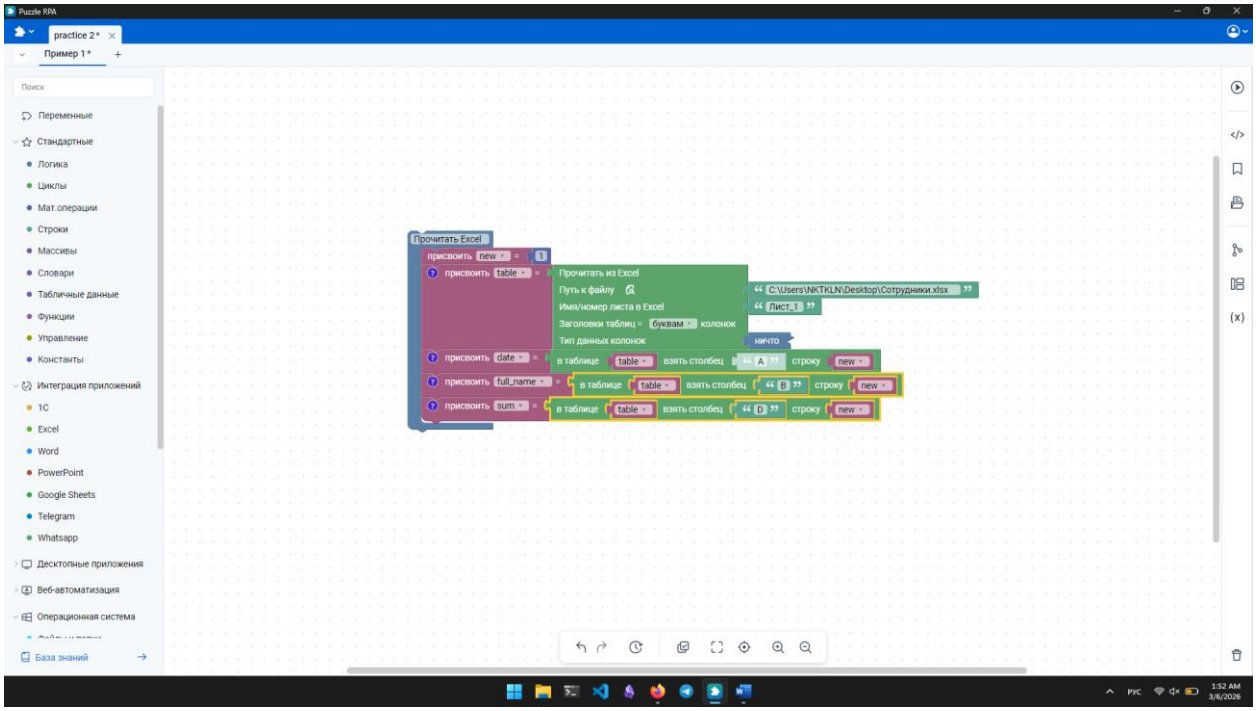


Присвоение переменной данные из конкретной ячейки





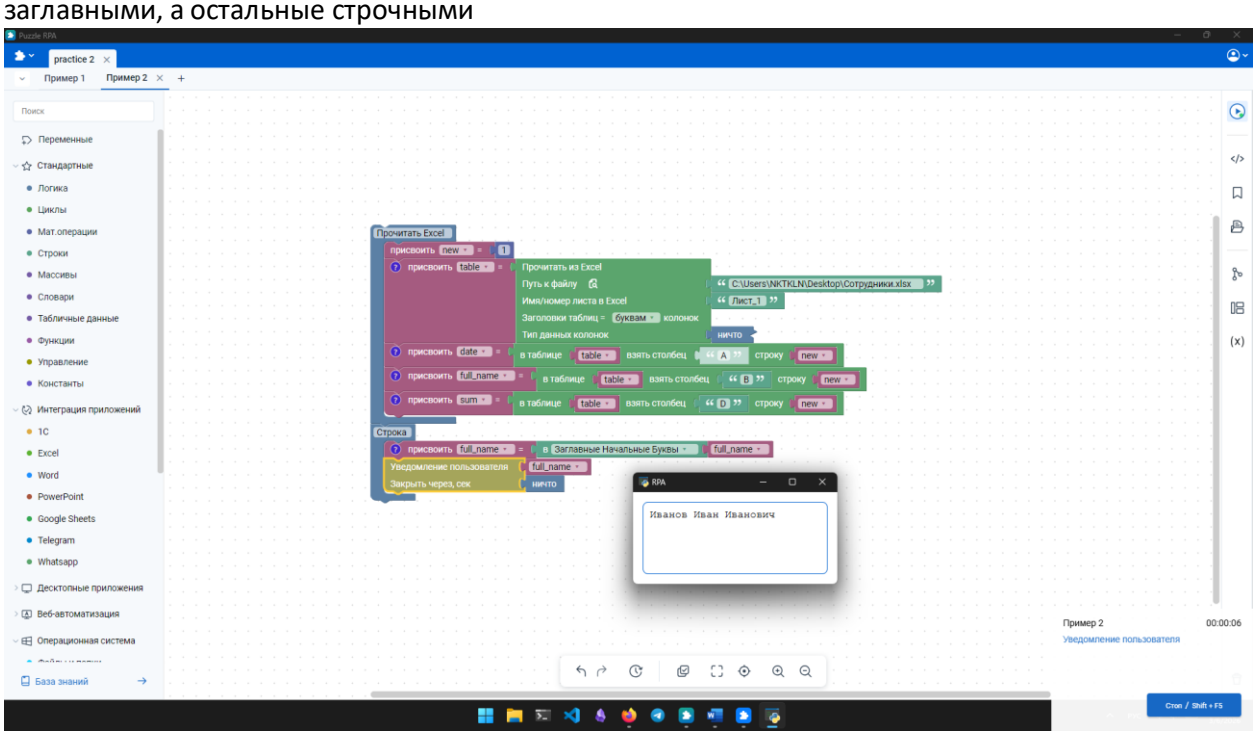
Группируем блоки чтения первой строки из таблицы



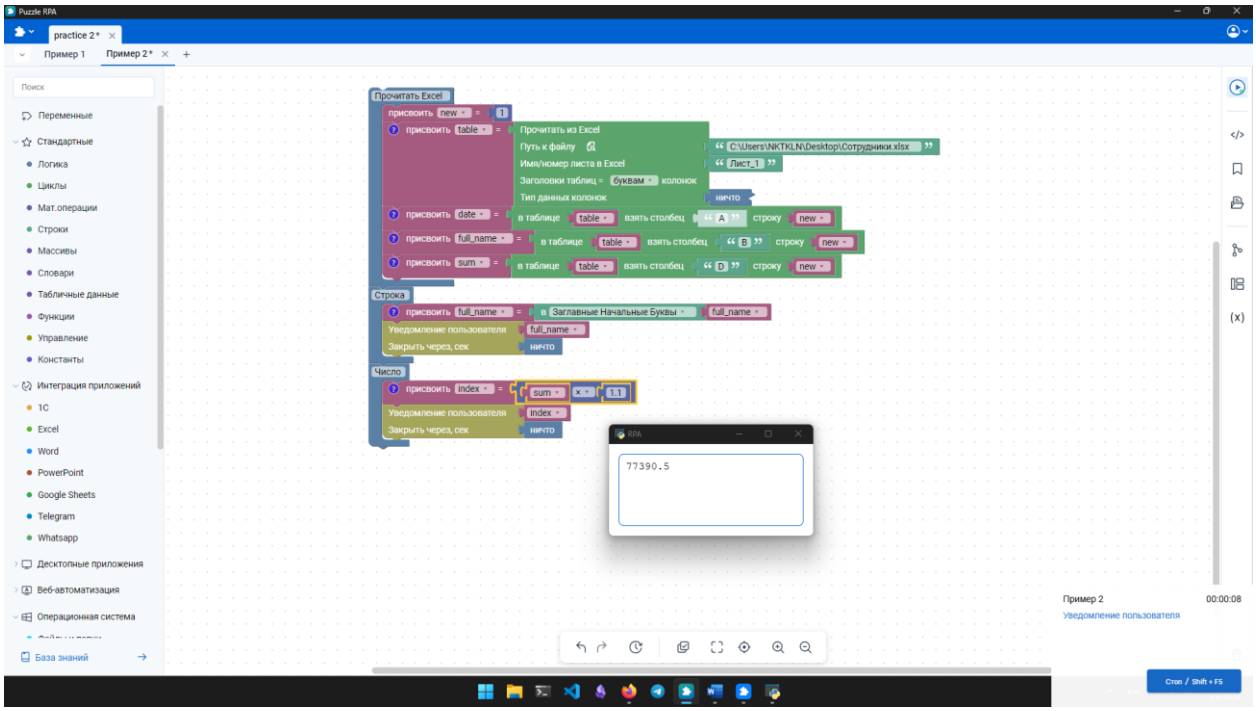
В итоге мы получаем процесс чтения строки под задаваемым номером из заданного файла

Пример 2. Работа с типами данных

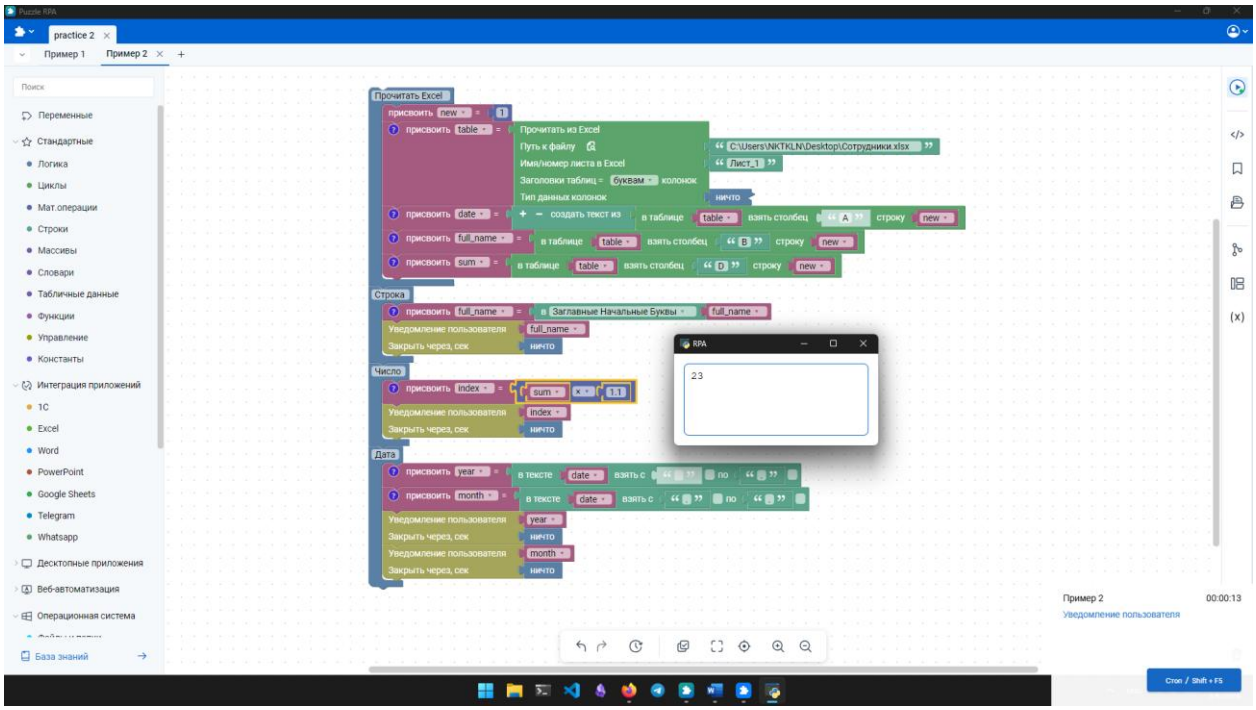
Работа со строками, а именно форматирование ФИО, чтобы все начальные буквы были заглавными, а остальные строчными



Увеличение суммы на 10%

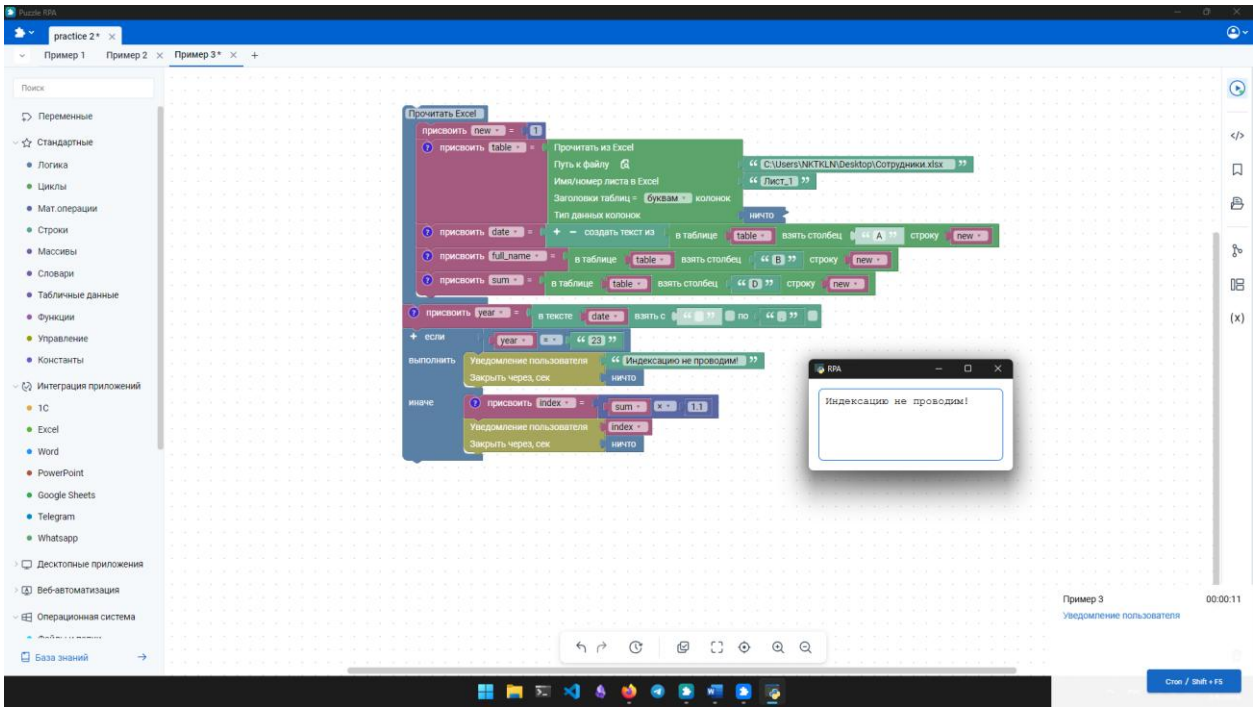


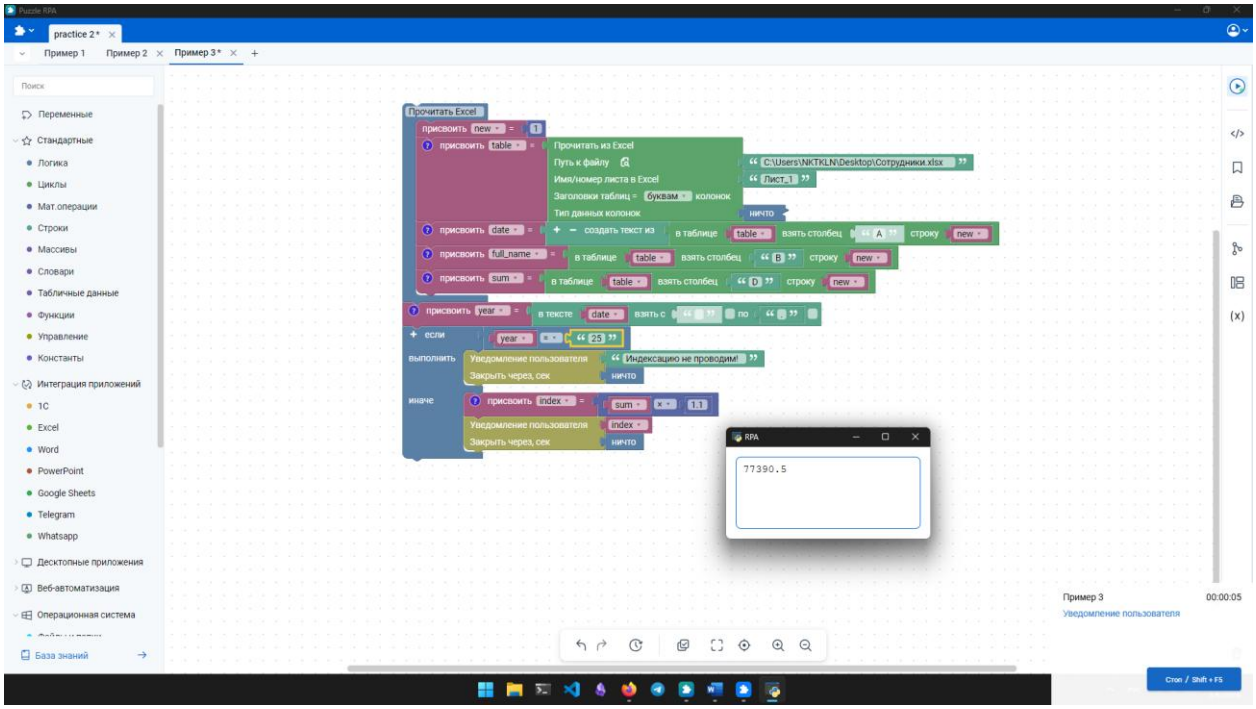
Распарсить дату, используя строковые операции



В ходе данного примера мы научились работать с различными операциями над типами данных.

Пример 3. Условие (Если)

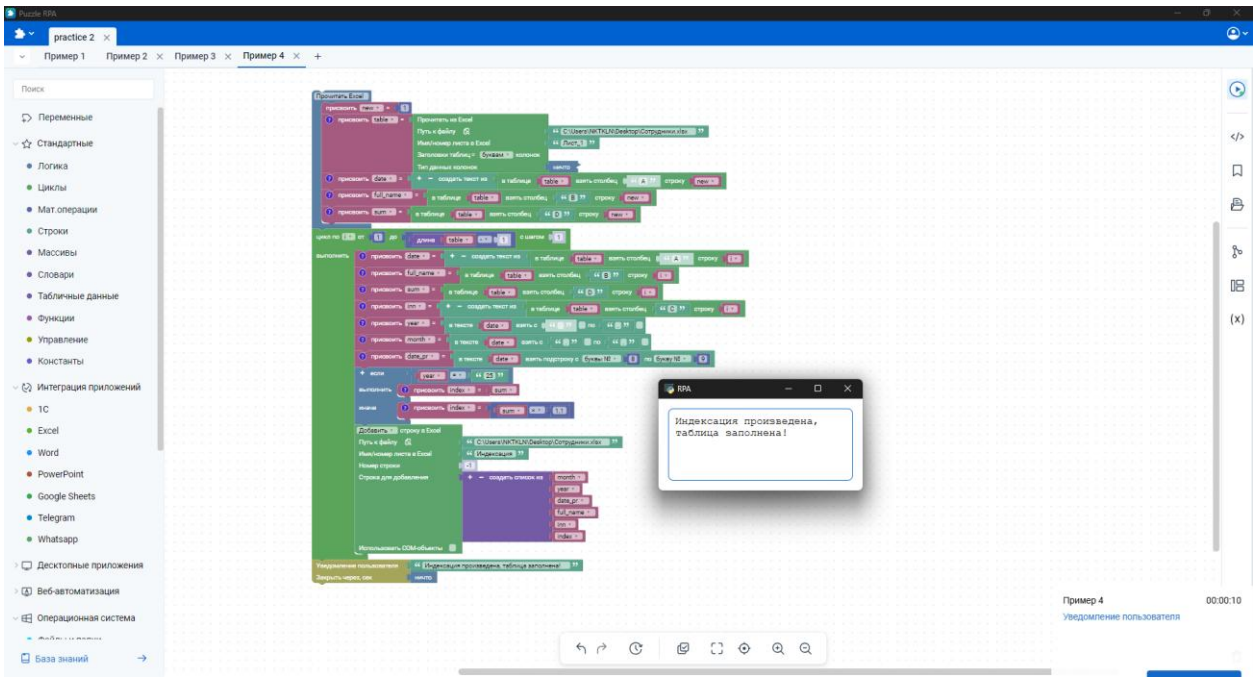


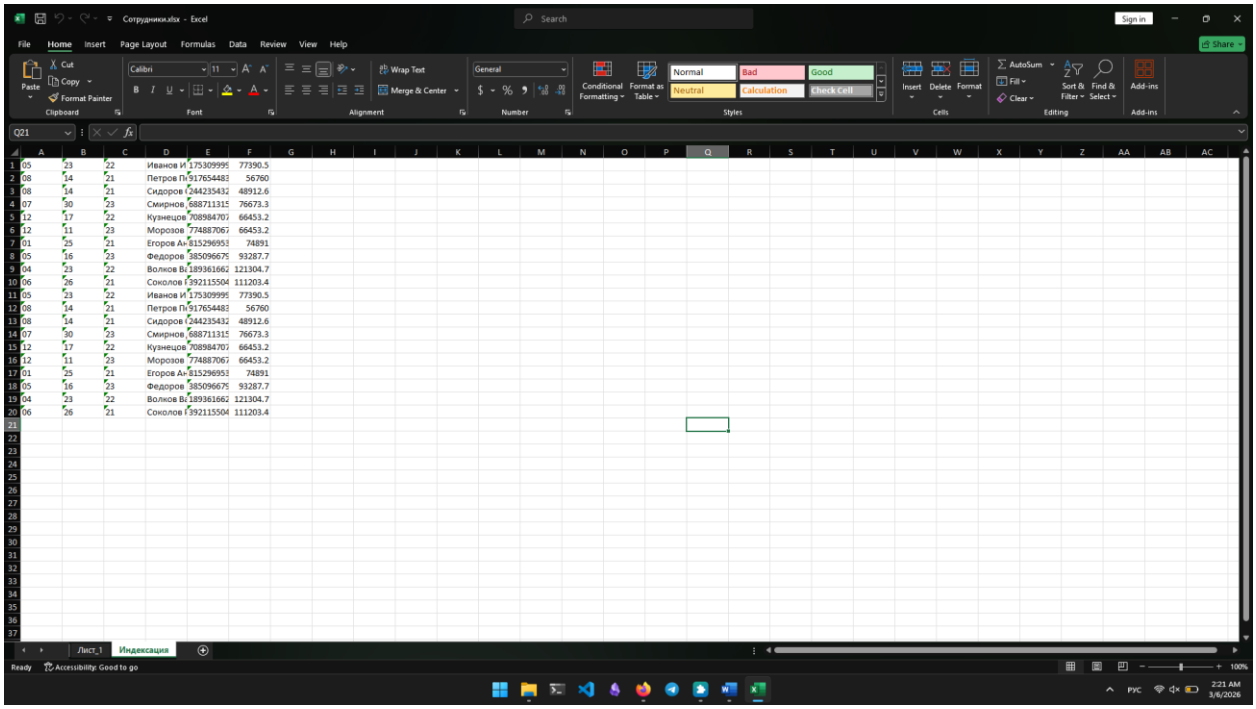


В ходе примера был реализован процесс с условным блоком, который работает по следующему алгоритму:

```
Если Истина (год = 2025)
→ Уведомление:
"Индексацию не проводим!"
Иначе
→ Выполняем блоки индексации
→ Показываем значение index
```

Пример 4. Алгоритм для всей таблицы





В ходе выполнения примера, был реализован процесс полной индексации таблицы сотрудников, используя циклы, условия и другие функции.

КОНТРОЛЬНЫЕ ЗАДАНИЯ

Вариант 11

Тематика: Онлайн-галерея для художников — анализ активности пользователей и популярности работ

Контекст задачи

Онлайн-галерея позволяет художникам загружать свои работы, отслеживать просмотры, лайки и комментарии, а пользователям — оценивать и сохранять понравившиеся произведения.

Менеджмент сталкивается с типичными проблемами:

- часть работ получает мало просмотров или низкие оценки;
- отдельные художники редко обновляют свои портфолио;
- необходимо выявлять популярные и проблемные работы, а также активных пользователей;
- прогнозировать, какие работы могут стать популярными в будущем.

Цель кейса: реализовать RPA-процесс, который автоматически анализирует активность пользователей и просмотры работ, рассчитывает ключевые показатели популярности, классифицирует работы и пользователей по уровню риска невовлечённости, формирует итоговый отчёт в Excel и выводит уведомление в модальном окне.

Задание 1. Анализ активности за январь (базовый уровень)

Выполняется в Puzzle RPA и Excel.

История: Вы — аналитик онлайн-галереи. Необходимо оценить активность художников и пользователей за первый месяц года, выявить работы с низкой популярностью и составить управленческую сводку.

---

1. Подготовка данных (Excel)

Создать файл Gallery_Activity_Jan.xlsx с листом Works_Jan.

Структура таблицы:

Поле	Тип	Смысл
WorkID	string	Уникальный идентификатор работы
ArtistName	string	Имя художника

Поле	Тип	Смысл
UserName	string	ФИО пользователя, оставившего отзыв
Views	decimal	Количество просмотров
Likes	decimal	Количество лайков
Comments	int	Количество комментариев
Rating	decimal	Средняя оценка работы пользователями 1–5
UploadDate	date	Дата загрузки работы

Методический комментарий: таблица отражает реальные действия пользователей и показатели популярности работ.

---

2. Инициализация переменных и счётчиков (Puzzle RPA)

Счётчик	Начальное значение	Смысл
total_works	0	Общее количество работ
low_view_works	0	Работы с просмотрами < 50
low_rating_works	0	Работы с оценкой < 3
commented_works	0	Работы с комментариями
total_views	0	Суммарное количество просмотров
total_likes	0	Суммарное количество лайков

---

3. Преобразование данных (Puzzle RPA, на каждой строке таблицы)

- views, likes, rating → decimal
- comments → int
- uploaddate → date
- artistname, username → удалить пробелы
- workid → верхний регистр

Методический комментарий: корректное приведение типов обеспечивает точность вычислений и корректное сравнение значений.

---

4. Расчёты (Puzzle RPA, на каждой строке таблицы)

engagement_score = views + likes * 2 + comments * 3

popularity_percent = (engagement_score / max_engagement) * 100

- engagement_score — интегральный показатель вовлечённости для работы;
- popularity_percent — относительная популярность работы по сравнению с самой популярной работой.

---

5. Логика классификации работ (Puzzle RPA, на каждой строке таблицы)

Простые условия:

if views < 50:

    low_view_works += 1

if rating < 3:

    low_rating_works += 1

if comments > 0:

    commented_works += 1

Составные условия:

if popularity_percent < 30 OR rating < 3:

    risk_category = "At Risk"

else:

    risk_category = "Engaged"

Вложенные условия:

if popularity_percent < 10 AND rating < 2:

    risk_category = "Critical"

elif popularity_percent < 30 OR rating < 3:

    risk_category = "High Risk"

else:

    risk_category = "Stable"

Методический комментарий: простые условия фиксируют просмотры, лайки и комментарии, составные — выявляют риск низкой популярности, вложенные — определяют итоговую категорию работы.

---

6. Формирование аналитической таблицы (Excel)

Файл Gallery_Analysis_Result.xlsx, лист Analysis_Jan с полями:

| WorkID | ArtistName | Views | Likes | Comments | Rating | EngagementScore | PopularityPercent | RiskCategory |

---

7. Итоговое уведомление (Puzzle RPA, модальное окно)

f"""Анализ работ за январь завершён.

Всего работ: {total_works}

С низкими просмотрами: {low_view_works}

С низкими оценками: {low_rating_works}

С комментариями: {commented_works}

Суммарные просмотры: {total_views}

Суммарные лайки: {total_likes}

Отчёт сохранён в Gallery_Analysis_Result.xlsx (лист Analysis_Jan).

"""

Задание 2. Прогнозная оценка работ за февраль (расширенный уровень)

Выполняется в Puzzle RPA и Excel.

История: Во втором месяце необходимо оценить потенциальную популярность и риск низкой вовлечённости работ с учётом текущих просмотров, оценок и прогнозируемого интереса пользователей, используя цикл по строкам таблицы.

---

1. Подготовка данных (Excel)

Файл Gallery_Activity_Feb.xlsx, лист Works_Feb с дополнительными полями:

Поле	Тип	Смысл
WorkID	string	Уникальный идентификатор работы
ArtistName	string	Имя художника
Views	decimal	Количество просмотров
Likes	decimal	Количество лайков

Поле	Тип	Смысл
Comments	int	Количество комментариев
Rating	decimal	Средняя оценка работы пользователями 1–5
ForecastInterest	decimal	Прогнозируемый интерес пользователей 1–5
UploadDate	date	Дата загрузки работы

Методический комментарий: прогнозируемый интерес позволяет оценить будущую вовлечённость.

---

2. Инициализация переменных и счётчиков (Puzzle RPA)

Счётчик	Начальное значение	Смысл
total_works	0	Общее количество работ
low_view_works	0	Работы с просмотрами < 50
low_rating_works	0	Работы с оценкой < 3
commented_works	0	Работы с комментариями
total_views	0	Суммарное количество просмотров
total_likes	0	Суммарное количество лайков
total_forecast_risk	0	Суммарный прогнозный риск (баллы)

Методический комментарий: переменная total_forecast_risk суммирует баллы риска каждой работы:

- Critical → +3
- High Risk → +2
- At Risk → +1
- Stable → 0

---

3. Цикл по строкам таблицы (Puzzle RPA)

На каждой итерации:

1. Привести типы данных:

- views, likes, rating, forecastinterest → decimal
- comments → int

- uploaddate → date
- artistname, workid → удалить пробелы / верхний регистр

2. Вычислить показатели:

engagement_score = views + likes * 2 + comments * 3

popularity_percent = (engagement_score / max_engagement) * 100

forecast_diff = popularity_percent - (forecastinterest * 20)

- forecast_diff отражает разницу между фактической популярностью и прогнозируемым интересом.

---

4. Многоуровневая логика прогнозного риска (Puzzle RPA, на каждой строке)

```
if popularity_percent < 10 AND rating < 2:
    risk_category = "Critical"
    total_forecast_risk += 3
elif popularity_percent < 30 OR rating < 3:
    risk_category = "High Risk"
    total_forecast_risk += 2
elif popularity_percent < 50 OR forecastinterest < 3:
    risk_category = "At Risk"
    total_forecast_risk += 1
else:
    risk_category = "Stable"
```

Методический комментарий: выполняется для каждой строки. Баллы суммируются в total_forecast_risk, чтобы оценить общий прогнозный риск низкой вовлечённости на платформе.

---

5. Формирование результирующего Excel (Excel)

Файл Gallery_Analysis_Result.xlsx, лист Analysis_Feb с полями:

| WorkID | ArtistName | Views | Likes | Comments | Rating | EngagementScore | PopularityPercent | ForecastDiff | RiskCategory |

---

6. Итоговое модальное уведомление (Puzzle RPA)

```
f"""Прогнозный анализ работ за февраль завершён.
```

Всего работ: {total_works}

С низкими просмотрами: {low_view_works}

С низкими оценками: {low_rating_works}

С комментариями: {commented_works}

Суммарные просмотры: {total_views}

Суммарные лайки: {total_likes}

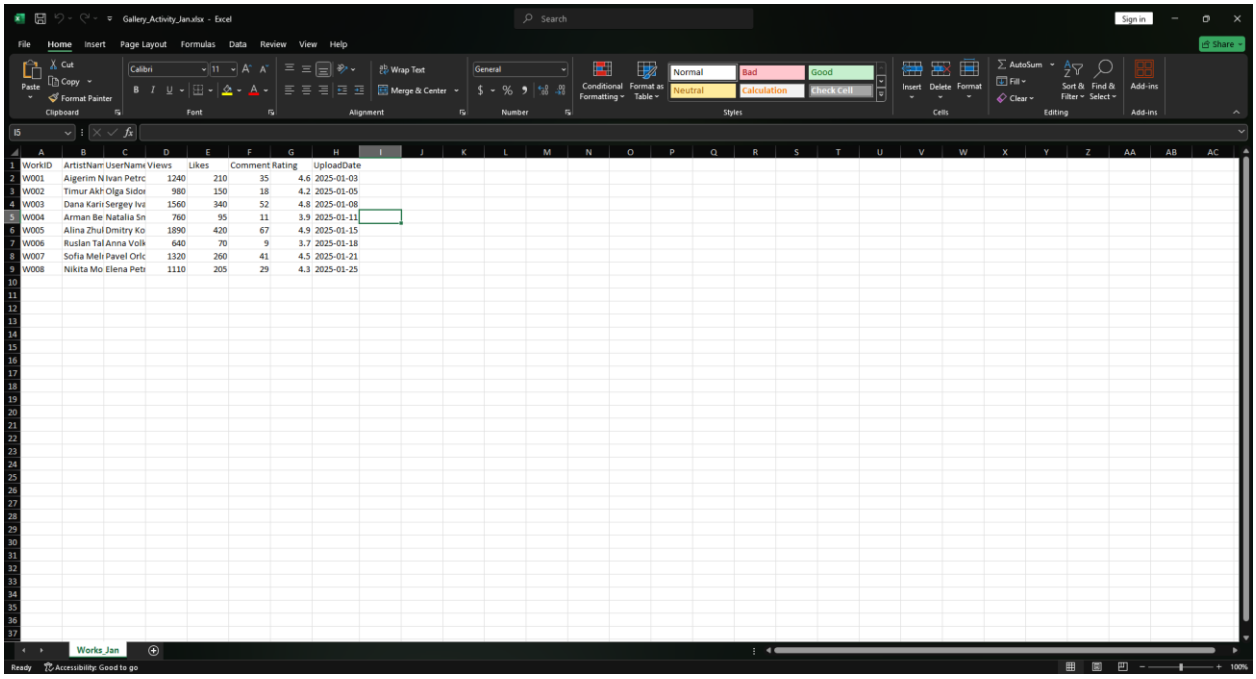
Суммарный прогнозный риск вовлечённости: {total_forecast_risk} баллов

Отчёт сохранён в Gallery_Analysis_Result.xlsx (лист Analysis_Feb).

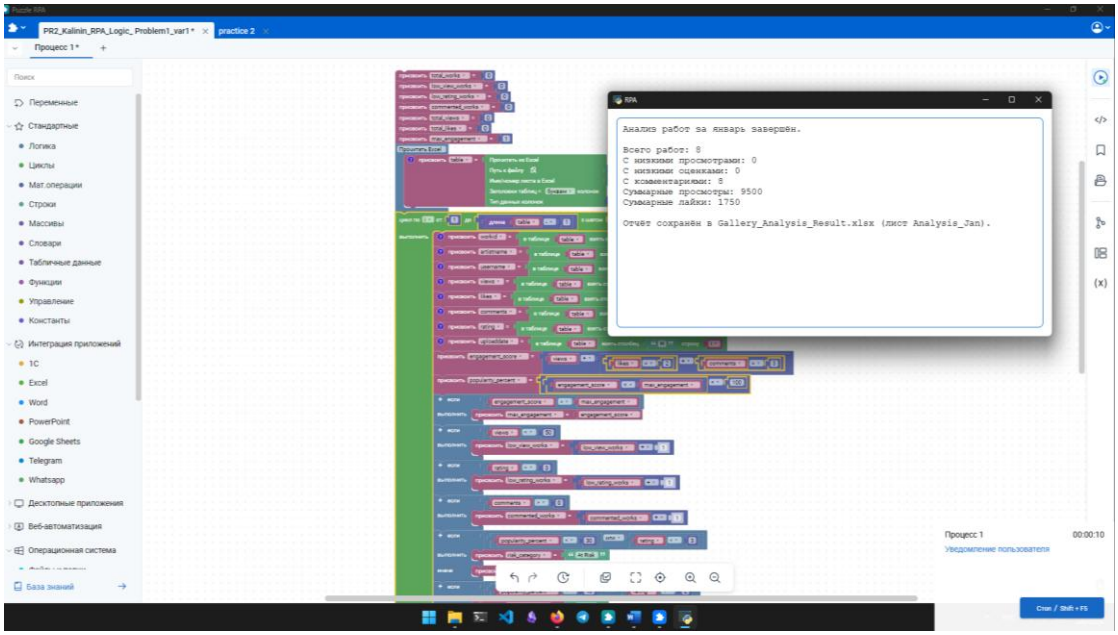
"""

Контрольное задание 1

Таблица с данными

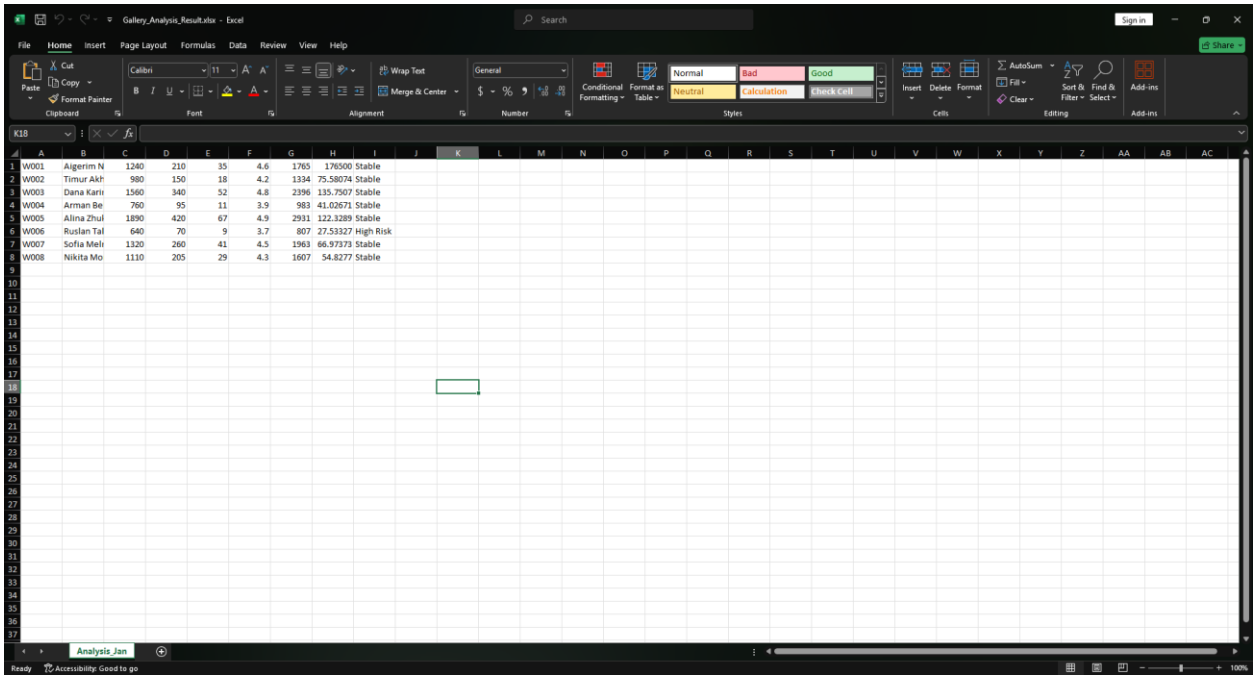


Успешное выполнение кода процесса



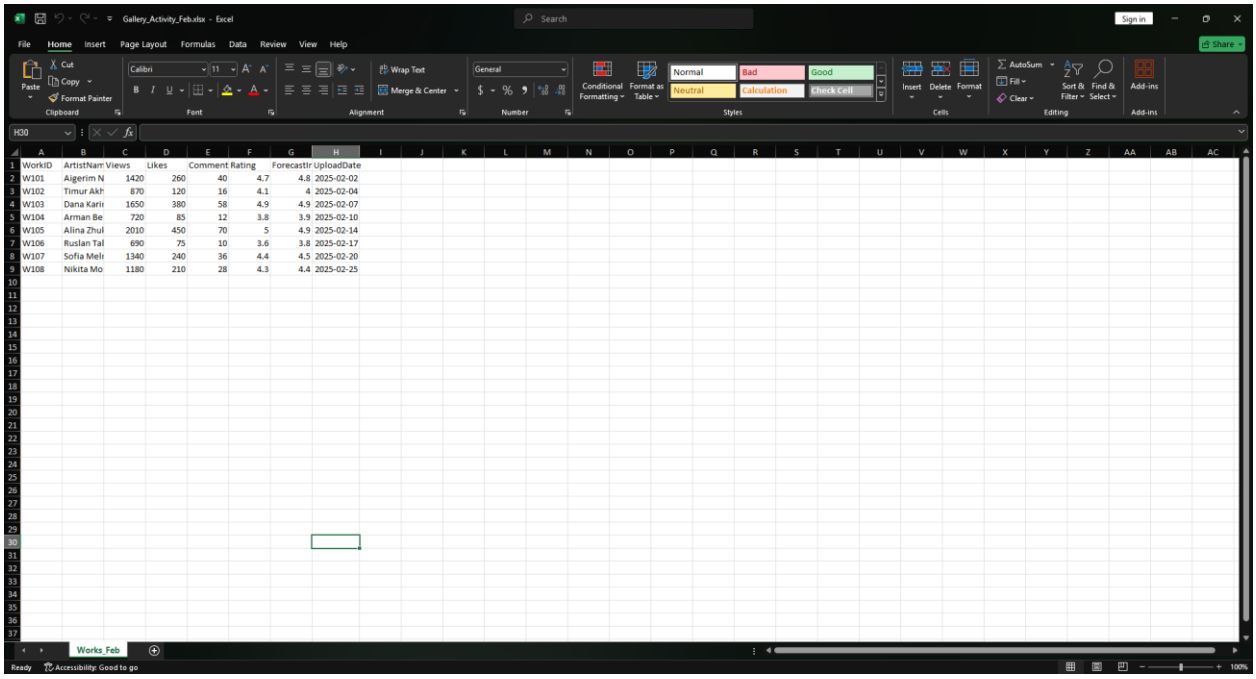
Алгоритм решения: загрузить данные из Excel, очистить и привести поля к нужным типам, затем для каждой работы рассчитать engagement_score и popularity_percent. После этого по условиям определить уровень риска (Critical / High Risk / Stable), одновременно подсчитать общую статистику по просмотрам, лайкам и проблемным работам. Результат — итоговая аналитическая таблица в Excel для выявления популярных и слабых работ.

Таблица с выходными данными

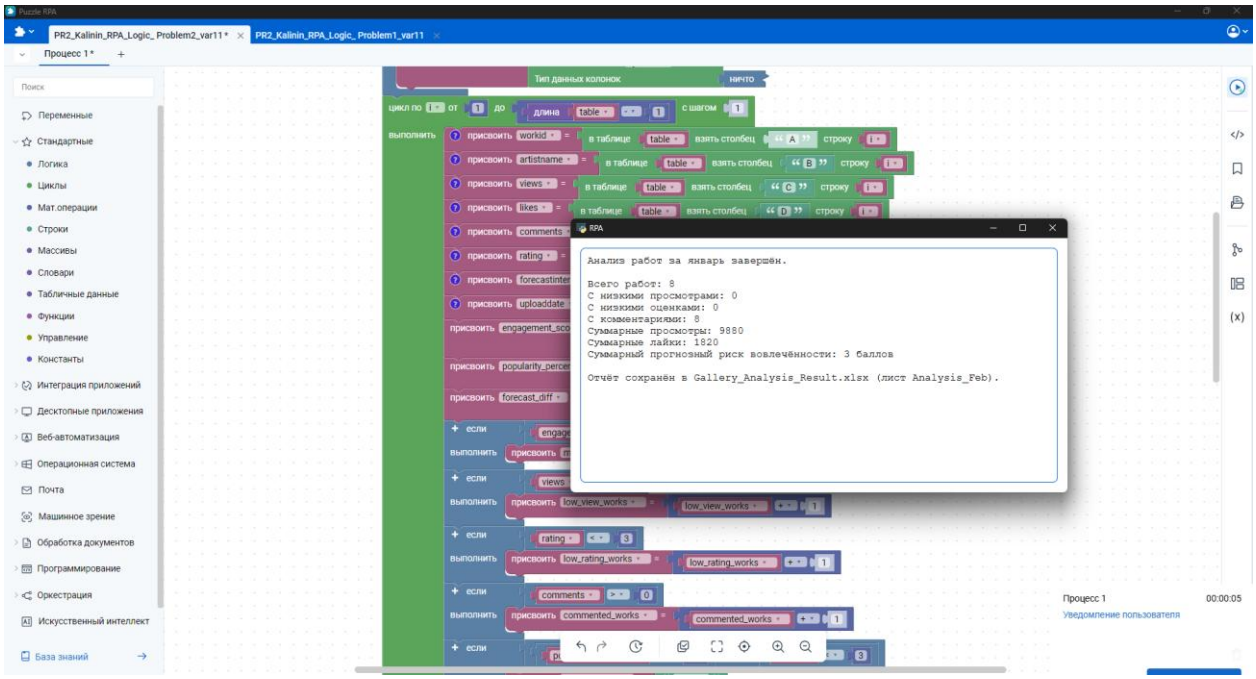


Контрольное задание 2

Таблица с данными



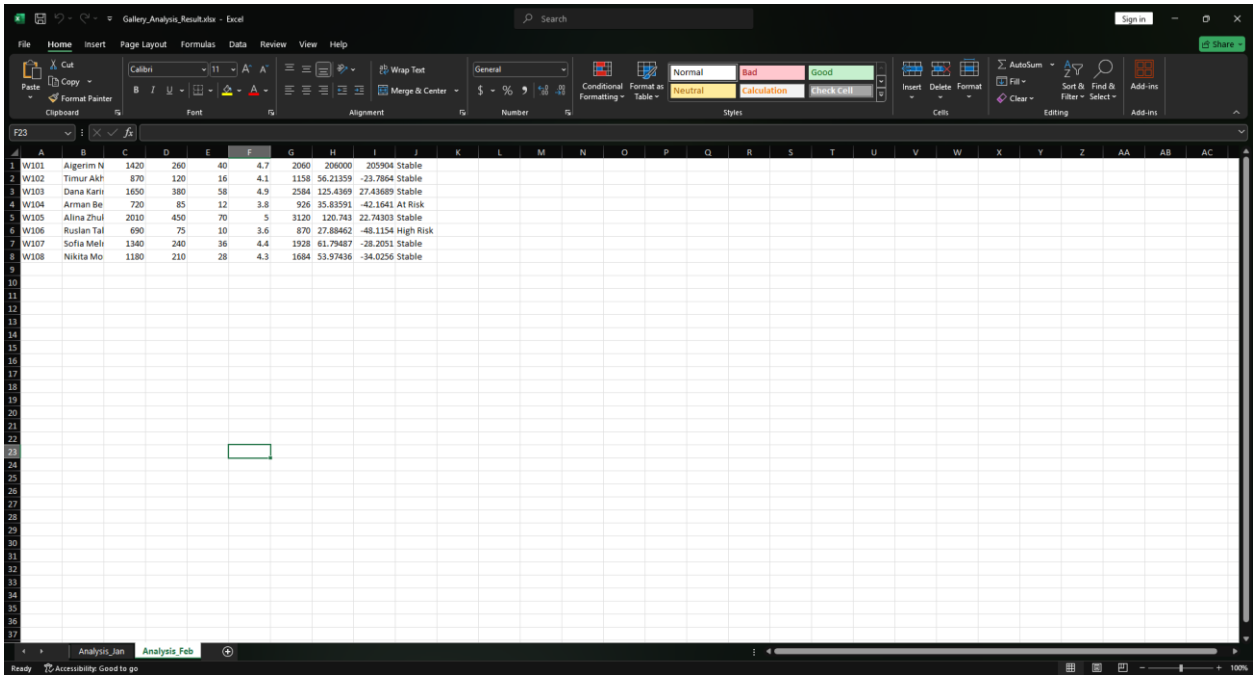
Успешное выполнение кода процесса



Для второго задания алгоритм дополняется прогнозной оценкой: после загрузки и очистки данных по каждой работе, кроме engagement_score и popularity_percent, рассчитывается forecast_diff — отклонение фактической популярности от прогнозируемого интереса. Далее в цикле каждая работа относится к одной из категорий риска (Critical / High Risk / At Risk / Stable), а её уровень риска переводится в баллы и суммируется в общий показатель total_forecast_risk. В результате

формируется Excel-отчёт с прогнозными метриками и итоговое уведомление с общей статистикой и суммарным риском вовлечённости за февраль.

Таблица с выходными данными



	A	B	C	D	E	F	G	H	I	J
1	W101	Aigerim N	1420	260	40	4.7	2060	206000	205904	Stable
2	W102	Timur Akh	870	120	16	4.1	1158	56.21359	-23.7864	Stable
3	W103	Dana Karii	1650	380	58	4.9	2584	125.4369	27.43689	Stable
4	W104	Arman Be	720	85	12	3.8	926	35.83591	-42.1641	At Risk
5	W105	Alina Zhul	2010	450	70	5	3120	120.743	22.74303	Stable
6	W106	Ruslan Tal	690	75	10	3.6	870	27.88462	-48.1154	High Risk
7	W107	Sofia Mel	1340	240	36	4.4	1928	61.79487	-28.2051	Stable
8	W108	Nikita Mo	1180	210	28	4.3	1684	53.97436	-34.0256	Stable

Ссылка на репозиторий: https://github.com/NKTKLN/mirea-puzzle-rpa/tree/main/practice_2